zkSketch: Verifiable and Privacy-Preserving Sketch-Based Analytics via Non-Interactive Zero-Knowledge Proofs

Anonymous Author(s)

Abstract

Regulated analytical workloads — periodic financial reporting, blockchain analytics, telemetry archives — publish aggregate answers (top-k, distinct counts, frequencies, quantiles) over privately committed data streams. Approximate sketches (Count-Min, HyperLogLog, SpaceSaving, Greenwald–Khanna) are the standard plaintext tool, and non-interactive zero-knowledge proofs over commitments are the natural cryptographic primitive for verifying answers without disclosing inputs. The two combine poorly under streaming workloads, because regenerating a ZK proof over the entire historical stream after every new committed batch defeats the central appeal of sketches: incremental maintenance.

We present zkSketch, the first non-interactive ZK framework for *streaming sketch maintenance*. zkSketch maintains a privately committed sketch alongside an efficiently updatable proof object: each new batch advances the committed state under a self-contained *transition proof* local to the batch and the previous committed state; per-epoch proofs are composed online via recursive-SNARK composition into a single history root checkable in $O(1)$ work, under three composition topologies (chained / folding, binary transition tree, k-ary batched tree) with explicit trade-offs against batch size, parallelism, and per-core memory; and each transition circuit certifies the batch's *net effect* on the committed sketch via a sortedness-and-segment-aggregation *delta certificate*, rather than replaying the sketch's plaintext random-write procedure inside the circuit. The framework instantiates on the five canonical sketch families through a seven-slot interface, and four compositional techniques (SDC, HPP, NDC, CCS) close the gap to a deployment-grade service.

Keywords

zero-knowledge proofs, approximate query processing, sketches, verifiable databases, streaming, recursive SNARKs

1 Introduction

Setting. Regulated analytical workloads publish aggregate answers over privately committed data streams. Periodic financial filings report top-k counterparties and distinct-account counts over a committed transaction log; blockchain-analytics indexes report heavy hitters and frequency aggregates over committed blocks; telemetry archives report quantiles and cardinalities over vendor-signed event streams. The raw records are sensitive and cannot be shown to third-party auditors, but the published aggregates must still be verifiable end-to-end against the committed inputs. Compact summaries — Count-Min [8], HyperLogLog [9], SpaceSaving [22], Greenwald–Khanna [12] — are the plaintext tool of choice for the aggregates, and non-interactive zero-knowledge proofs over committed data [2, 14, 19, 41] are the natural cryptographic primitive for verifying answers without disclosing the inputs.

Central challenge: incremental, not from scratch. The central challenge in verifiable sketch analytics is *not* merely to prove a single sketch computation in ZK. Real analytical streams evolve continuously: new records arrive over time, the committed sketch state changes, and queries may be issued before the stream ends. Recomputing a ZK proof over the entire historical stream after every new batch would destroy the central appeal of sketches — their incremental nature in the plaintext world. We therefore ask:

> *Can a single untrusted prover maintain a privately committed sketch together with an efficiently updatable proof object, so that each new batch only requires a local transition proof, while auditors can still verify the entire committed history succinctly at any time?*

Three technical sub-problems. This framing crystallises into three concrete sub-problems that organise the rest of the paper.

(C1) Incremental verifiability. After epoch B_t arrives, the prover should produce a proof for the *transition* $(S_{t-1}, B_t) \to S_t$, not a proof that $S_t = A_\tau(B_1, \ldots, B_t)$ rebuilt from the empty initial state. Update-local proving means the per-batch prover cost depends mainly on the new batch and the changed sketch cells, not on the full history length.

(C2) Succinct history binding. Even with per-batch transition proofs $\pi_1, \ldots, \pi_t$ in hand, a verifier should not be required to check t proofs every time it audits. The system needs an updatable history accumulator that absorbs each new π_t and exposes one compact root commitment $R^{(t)}$ checkable in work independent of t. Recursive SNARKs, folding accumulators, and proof trees are the candidate primitives.

(C3) Efficient transition certification. The per-batch transition circuit must itself be cheap. A direct in-circuit replay of the plaintext sketch update procedure is expensive: sketch updates evaluate data-dependent hashes, write to private addresses determined by those hashes, handle within-batch collisions, and rebind a committed state after each write — the worst-case operation pattern for PLONKish backends. The transition circuit must therefore avoid replaying the procedure and instead certify only the batch's *net effect* on the committed state.

Two further concerns shape the same design space and are addressed jointly throughout the paper: *(C4) version-correct query answering* — a query proof must be tied to the committed sketch version containing exactly the uploaded prefix visible before the query — and *(C5) leakage control* — public artefacts such as the circuit shape and the committed capacities must not reveal private distributional information such as touched-cell counts or key skew.

Our approach. zkSketch realises the framing above in three layers.

Per-epoch transition proofs (§4, addressing C1 and C3) certify how one committed batch advances the committed sketch state.

The transition circuit verifies a *delta certificate* — a sorted, segment-aggregated patch over only the cells the batch actually changes — instead of replaying the sketch's hash-determined random writes. The certificate's shape depends only on public capacities, so the circuit is data-oblivious.

Recursive history composition (§4.3, addressing C2) absorbs each new per-epoch proof into a single history root commitment $R^{(t)}$. The root proof $\Pi^{(t)}$ is verifiable in $O(1)$ work irrespective of the number of epochs. We develop the composition design space as three topologies — chained / folding for line-rate ingest, binary transition tree for parallel audit-window provers, and k-ary batched tree for throughput-oriented deployments — and characterise their trade-offs against per-batch latency, prover memory, parallelism, and per-core budget.

Sketch instantiations and deployment-grade mechanisms (§5–§**??**, addressing C3, C4, and C5) instantiate the framework on the four sketch families through a seven-slot interface (Π_τ) and a two-pattern dichotomy (Pattern A linear / Pattern A max / Pattern B), tie every query proof to a committed sketch version via epoch-closure boundaries, and add four compositional techniques — shared delta codomain (SDC), hash-profile pre-commitment (HPP), negative-delta certificates (NDC), and the capacity co-sketch (CCS) — that amortise sort and hash work across coupled sketches and batches, extend Pattern A linear sketches to delete-aware updates, and bound capacity leakage under a differentially private mechanism.

Contributions.

- **Streaming sketch maintenance framework.** A committed sketch object that advances version-by-version under self-contained per-epoch transition proofs and an updatable history root, bound via recursive-SNARK composition into one root commitment $R^{(t)}$ summarising the verifiable streaming history. Merge proofs for independently maintained shards are a separate first-class operation.
- **Composition design space.** Three composition topologies — chained / folding, binary transition tree, k-ary batched tree — expose one $O(1)$ verifier interface but differ on online latency, per-batch maintenance cost, parallelism, and per-core memory; we characterise their regimes against deployment knobs.
- **Per-batch transition circuits across four sketch families.** A seven-slot interface and a two-pattern dichotomy instantiate Count-Min, HyperLogLog, SpaceSaving, and Greenwald–Khanna under one circuit pipeline organised around a verification-over-computation discipline.
- **Deployment-grade system mechanisms.** Four compositional techniques (SDC, HPP, NDC, CCS), a layered architecture with a pluggable certificate-shape catalog, a deployment-aware compiler over a three-dimensional cost surface (prover × verifier × leakage), and workflow decomposition turn the per-circuit baseline into a deployable service.

Scope. ZK certifies exact adherence to the *declared sketch algorithm*, not exact equality with the true answer over the underlying dataset; approximation error is inherent to sketch theory. Input completeness — that every real-world event reached the sketch — requires an authenticated input log on the data-owner side and is outside the cryptographic scope of zkSketch.

Roadmap. §2 fixes preliminaries. §3 defines the system model, threat model, and committed sketch object. §4 develops the streaming sketch maintenance framework: per-epoch transition proofs, the long-history bottleneck, recursive interval proofs, and three composition topologies. §5 instantiates the framework circuit-by-circuit on Count-Min and Greenwald–Khanna as the two pattern representatives; HyperLogLog and SpaceSaving appear in the appendix. §**??** addresses deployment-grade system mechanisms. §6 describes the planned evaluation, §7 discusses related work, and §9 concludes.

2 Preliminaries

2.1 Zero-Knowledge Proofs (ZKPs)

Zero-knowledge proofs (ZKPs) allow a prover to convince a verifier that a public statement is true without revealing any information beyond the validity of the statement itself. ZKP protocols are commonly classified as either interactive [**?**] or non-interactive [**?**]. For certain public-coin interactive protocols [**?**], the Fiat–Shamir transformation [**?**] can be used to turn interactive ZKPs into non-interactive ZKPs. In this work, we focus on non-interactive ZKPs, where the proof is consolidated into a single message that can be verified later by any verifier. This makes such proofs particularly suitable for asynchronous settings in which the prover and verifier need not be online at the same time. More specifically, we use *zero-knowledge succinct non-interactive arguments of knowledge* (ZK-SNARKs) [**?**], which produce short proofs and support fast verification properties that are well suited to the offline-checkable model. A ZK-SNARK provides three standard guarantees:

- *Completeness*: an honest prover can convince an honest verifier of any true statement.
- *Knowledge soundness*: a dishonest prover cannot convince a verifier of a false statement except with negligible probability, and an extractor recovers the witness from any successful prover.
- *Zero-knowledge*: the verifier learns nothing about the witness beyond the validity of the statement.

At a high level, a ZK-SNARK proves knowledge of a private witness that satisfies a publicly specified relation for a given public statement. The witness contains the hidden information needed to justify why the statement is true; depending on the application, it may include secret inputs, intermediate values, or an execution trace of the computation being certified. To enable efficient proving and verification, the target computation is encoded as an arithmetic circuit, or equivalently as a system of algebraic constraints over a finite field. This circuit is public and specifies how the public inputs and private witness must relate. In practice, the computation is represented by constraints, often expressed as low-degree polynomial equations, that are satisfied exactly when the witness corresponds to a valid execution of the computation. The prover then generates a succinct proof that it knows a witness satisfying all of these constraints, without revealing the witness itself.

Shall we introduce ZKSNARK and ZKSTARK here?

3 System Model and Problem Statement

zkSketch targets streaming analytical workloads in which raw records are private, but aggregate sketch answers must be verifiable.

As in plaintext stream processing, the logical input is potentially unbounded: records may arrive indefinitely, and clients may issue queries at any time during the stream. Unlike plaintext stream processing, however, the prover must convince an offline verifier that every published answer is consistent with the committed input stream, without revealing the raw records.

We model the input as an operation stream

$$\mathcal{X} = \big((o_1, x_1), (o_2, x_2), \ldots, (o_t, x_t), \ldots\big),$$

where $o_t \in \{\text{upload}, \text{query}\}$ is the operation type at logical time t. If $o_t = \text{upload}$, then x_t is a data record. If $o_t = \text{query}$, then x_t is the query parameter. The meaning of x_t depends on the query type: for a point query it is a key, for a range query it is a range, and for a parameter-free query such as distinct count it is set to a dummy value $\perp$.

For a query operation (o_t, x_t), the intended output is evaluated over all uploaded records that appeared before time t:

$$F_\tau\left(\{x_i : i < t,\ o_i = \text{upload}\}, x_t\right),$$

where F_τ is the query semantics induced by sketch family τ. For example, F_τ may be frequency estimation for Count-Min, distinct counting for HyperLogLog, top-k heavy hitters for SpaceSaving, or rank/quantile estimation for Greenwald–Khanna. Since these are sketch-based algorithms, the output is the declared sketch answer, not necessarily the exact answer over the raw dataset.

The design goal is the streaming analogue of bounded-cost sketching: the prover's memory and per-record work should be bounded, or grow only mildly with the stream length, while the verifier obtains transferable non-interactive proofs of correctness. The cost of a query should depend on the sketch size and proof objects, not on scanning the entire stream.

3.1 Actors and Public Artifacts

There are three roles.

Data owners. Data owners produce upload records. For each record, or for a group of consecutive records, the data owner publishes a binding commitment before the record is used for sketch maintenance. Data owners need not participate in proof generation after committing their data.

Prover. The prover maintains the committed sketch state and produces all non-interactive proofs. The prover is untrusted: it may try to drop uploads, reorder records, use the wrong sketch parameters, fabricate state transitions, or answer queries from an incorrect version.

Verifier. The verifier is an auditor, regulator, client, or third party who checks published proofs offline. The verifier sees public commitments, metadata, answers, and proofs, but not the raw records or private sketch states. The verifier may be curious, so proof artifacts should not reveal more than the validity of the certified statements and the declared public parameters.

The public log contains four kinds of artifacts:

$$\mathsf{Log} = \left\{C_{B_t},\ \mathsf{Com}(S_t),\ R^{(t)},\ \pi\right\}_{t \geq 0}.$$

Here C_{B_t} is a commitment to a consecutive block of uploaded records, $\mathsf{Com}(S_t)$ is a commitment to the sketch state after processing that block, $R^{(t)}$ is a root commitment binding the update history up to version t, and π denotes the update, history, decode, merge, or release proofs published by the prover.

3.2 Epochs as Proof Granularity

The stream is logically item-wise, but proof generation is performed at a tunable *epoch* granularity. Let

$$u_1, u_2, \ldots$$

be the subsequence of uploaded records extracted from the operation stream. zkSketch partitions these records into consecutive epochs

$$B_t = (u_{\ell_{t-1}+1}, \ldots, u_{\ell_t}),$$

where the boundary ℓ_t may be chosen by record count, wall-clock time, memory pressure, or deployment policy. The special case $|B_t| = 1$ gives item-by-item proofs. Larger epochs amortize commitment openings, sorting, hash checks, delta aggregation, and proof generation across many records.

Epoching is therefore not an assumption about the data source. It is a systems knob that controls the latency–amortization tradeoff. Smaller epochs give fresher committed states and lower query latency. Larger epochs reduce amortized proving cost.

Queries may be issued at any logical time. To preserve the semantics that a query is evaluated over all uploads so far, zkSketch treats a query as a version boundary: before answering a query at time t, all uploads with timestamp less than t must be included in some committed epoch. If the current epoch is open, the prover closes it, publishes its commitment, and proves the corresponding state transition before emitting the decode proof. Thus the query answer is always tied to a versioned committed state containing exactly the uploaded prefix visible to the query.

3.3 Committed Sketch Object

For each registered sketch, zkSketch maintains a committed sketch object

$$\mathsf{Sketch}_t = (id,\ type,\ params,\ \mathsf{M},\ version,\ \mathsf{Com}(S_t),\ R^{(t)},\ policy).$$

The fields have the following roles.

Identity and algorithm metadata. The field id is a unique handle for the sketch. The field $type \in \{\mathsf{CM}, \mathsf{CS}, \mathsf{HLL}, \mathsf{SS}, \mathsf{GK}\}$ identifies the sketch family: Count-Min, HyperLogLog, SpaceSaving, or Greenwald–Khanna. The field $params$ records public dimensions such as the Count-Min depth and width (d, w), the HyperLogLog register count, or the SpaceSaving capacity s. The metadata M fixes the public algorithm choices, including the hash profile, counter representation, commitment scheme, and circuit capacities.

Cryptographic state. The commitment $\mathsf{Com}(S_t)$ binds the hidden sketch state at version t. The root $R^{(t)}$ binds the ordered history of committed epochs up to version t. Informally, $\mathsf{Com}(S_t)$ says "this is the current hidden sketch state," while $R^{(t)}$ says "this state is the result of the committed stream prefix." Section 4 constructs this root using recursive history composition.

Release policy. The field $policy$ records public post-processing rules for published answers, such as rounding, suppression below a threshold, or adding noise generated from a committed seed. Since

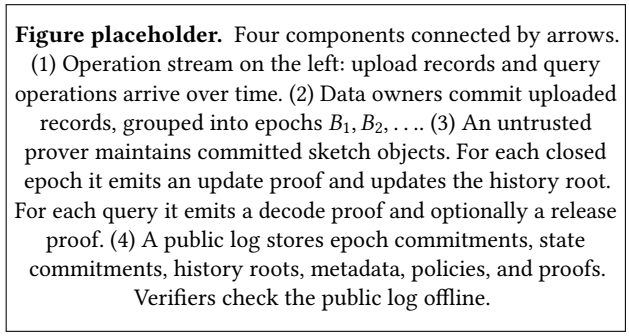


Figure 1: System model of zkSketch. The logical input is an operation stream of uploads and queries. Epochs are the proof granularity used to amortize ZK costs; query operations are answered against the committed stream prefix visible at the query time.

the policy is public, a release proof can certify that it was applied correctly.

3.4 System API

The committed sketch object supports four operations. Each operation produces a non-interactive proof whose public inputs are the relevant commitments, metadata, and declared parameters.

UpdateEpoch. Given a committed sketch object at version $t-1$ and a commitment C_{B_t} to the next epoch, the prover outputs a new state commitment $\mathsf{Com}(S_t)$, an update proof π_t, and an updated history root $R^{(t)}$. The proof certifies

$$S_t = A_\tau.\mathsf{UpdateEpoch}(S_{t-1}, B_t),$$

where epoch update means sequentially applying the ordinary item update to every record in B_t.

Decode. Given a query parameter q and a committed sketch state $\mathsf{Com}(S_t)$, the prover outputs an answer *ans* and a decode proof π_q. The proof certifies

$$ans = A_\tau.\mathsf{Decode}_q(S_t).$$

The state version t must include all upload records that precede the query in the operation stream.

Merge. Given two independently maintained compatible sketch objects, the prover outputs a merged object and a merge proof. Merge is used for shards, tenants, partitions, or range summaries. It certifies type and parameter compatibility, coverage disjointness or declared mergeability, and correct application of the sketch-family merge operator. Merge is separate from temporal history composition: merge combines independent summaries, whereas history composition binds an ordered stream prefix.

Release. Given a decoded answer and a public release policy, the prover outputs a publishable answer and a release proof. The proof certifies that the policy was applied correctly, for example by enforcing deterministic rounding, suppression, or noise generation from a committed seed.

3.5 Threat Model and Security Goals

The prover may be malicious. It may attempt to omit committed uploads, reorder records or epochs, use an undeclared hash profile, fabricate a state commitment, answer from a stale version, or return a value that is not the declared decoder's output. The verifier may be curious and may try to infer private records from proof bytes, public capacities, circuit shape, query timing, or released answers.

zkSketch aims to provide the following guarantees.

Update soundness. If an update proof verifies, then the new committed state is the result of applying the declared sketch update algorithm to the previous committed state and the committed epoch. A malicious prover cannot produce an accepting proof for a fabricated epoch transition except with negligible probability.

History soundness. If a history root proof verifies, then the current committed state is reachable from the initial state by applying the ordered committed epochs. Dropping an epoch, reordering epochs, replacing an epoch commitment, or substituting an intermediate state changes the history root except with negligible probability.

Query soundness. If a decode proof verifies for a query operation at logical time t, then the answer is the declared sketch decoder's output on a committed state that includes all uploads before t. If a release proof verifies, then the published value is obtained by applying the declared release policy to that decoded answer.

Zero knowledge and circuit obliviousness. The zero-knowledge property hides private witnesses, including raw records, sketch states, intermediate tables, sorted certificates, and commitment openings. In addition, the public circuit shape depends only on declared public capacities and parameters, not on the actual records or the actual set of touched sketch cells. If capacities are chosen adaptively from the data, zkSketch can release capacity values through a differentially private capacity co-sketch.

Throughout the paper, we assume that the underlying NIZK is complete, knowledge-sound, and zero-knowledge; commitments are computationally binding and, when required, hiding; and the hash function is collision-resistant. Input completeness—the property that every real-world upload is included in the committed operation stream—is the data owner's responsibility and is outside the cryptographic scope of zkSketch.

3.6 Problem Statement

Let

$$\mathcal{U}_{<t} = (u_1, \ldots, u_{N(t)})$$

be the sequence of uploaded records that occur before a query at logical time t. Let A_τ be the declared algorithm for sketch family τ, and let S_0 be the initial empty sketch state. The intended state for the query is

$$S_{N(t)} = A_\tau.\mathsf{Update}\big(\cdots A_\tau.\mathsf{Update}(A_\tau.\mathsf{Update}(S_0, u_1), u_2), \ldots, u_{N(t)}\big).$$

Equivalently, if the uploaded prefix has been partitioned into committed epochs $B_1, \ldots, B_r$, then

$$S_r = A_\tau.\mathsf{UpdateEpoch}\big(\cdots A_\tau.\mathsf{UpdateEpoch}(S_0, B_1), \ldots, B_r\big).$$

For a query parameter q, the prover must output a public answer *ans* and transferable proofs convincing any verifier that

$$ans = A_\tau.\mathsf{Decode}_q(S_r),$$

where S_r opens the public commitment $\mathsf{Com}(S_r)$ and is the result of the ordered committed upload prefix visible to the query.

The guarantee is with respect to the declared sketch algorithm. zkSketch does not certify that *ans* equals the exact answer over the raw dataset; sketch approximation error is inherent to the chosen sketch family. What zkSketch certifies is faithful execution of the declared sketch over the committed input stream.

Example 3.1 (Running example: a regulated transaction stream). *A regulated finance-data operator receives transaction records continuously and periodically answers aggregate queries. Consider the following five uploaded records:*

i	k_i *(counterparty)*	v_i *(amount)*
1	*ACME*	*100*
2	*BETA*	*50*
3	*ACME*	*200*
4	*GAMMA*	*75*
5	*BETA*	*30*

These records may arrive one by one, but the deployment may choose to place them in one epoch B_t for proof efficiency. For Count-Min we use $d = 2$ rows and $w = 4$ buckets. For Greenwald–Khanna we use the capacity parameter $s = 2$, summarising the empirical value distribution at a coarse rank error suitable for illustration; Appendix G reuses the same records for SpaceSaving with capacity $s = 2$. The same records will reappear in later sections to illustrate delta certificates and sketch-specific circuits.

4 zkSketch Framework: Verifiable Streaming Sketch Maintenance

We introduce the framework of zkSketch in this section. In the streaming data setting, data updates arrive continuously while the prover operates under bounded memory and computation. The prover therefore cannot wait until the entire stream has been observed before producing a ZKP: the stream is potentially unbounded, and any client query issued in the meantime would have to wait. Our framework adopts the *incremental update* idea: the prover emits proofs as updates arrive, so that when new updates come in, proof generation builds on the proofs already established rather than starting from scratch. This significantly reduces the latency a client sees when querying the data statistics based on the current sketch state, since at any moment the prover already holds an up-to-date proof of all updates processed so far.

In the following, Section 4.1 introduces the most direct realisation (one ZKP per update) and exposes the verifier-side $\Theta(E)$ bottleneck it leaves open. Section 4.2 collapses the chain of per-update proofs into a single root proof via recursive interval proofs, fixing the merge atom and the interval-proof contract. Section 4.3 schedules these merges over a streaming history as a forest of perfect subtrees, structurally analogous to a Merkle Mountain Range [35] but used only to organise the recursive composition. Section 4.4 handles queries interleaved with updates. The end-to-end soundness and privacy guarantees of the framework are stated in Appendix A.

4.1 Per-update proof chains and their cost

To process the stream, the most direct strategy is to generate one ZK proof per update. Whenever a new update b_i arrives, the prover produces a proof π_i certifying that the committed sketch state was advanced by exactly b_i.

We formalize one such proof as a per-update transition proof. Each π_i has public input

$$x_i \;=\; \big(\mathsf{Com}(S_{i-1}),\; C_{b_i},\; \mathsf{Com}(S_i),\; \mathsf{M}\big),$$

where M is the public metadata that fixes the sketch parameters, hash profile, counter representation, commitment scheme, and circuit capacities. The private witness w_i contains the old state, the opening of b_i, the new state, and any auxiliary witness required by the update circuit of Section 5. The proof certifies membership in the update relation

$$\mathcal{R}^{\tau}_{\mathsf{upd}} \;=\; \{(x_i, w_i) \;:\; S_i = A_\tau.\mathsf{Update}(S_{i-1},\, b_i)\}\,,$$

where τ identifies the sketch family.

The proof π_i is self-contained: it verifies against x_i alone, without reading past or future updates, so the prover can emit $\mathsf{Com}(S_i)$ and π_i online as soon as b_i is processed. Consecutive proofs are chained through their boundary state commitments: the next proof π_{i+1} takes $\mathsf{Com}(S_i)$ as its previous-state commitment. We call the resulting sequence the *per-update proof chain*.

A verifier that wishes to check all updates have been correctly applied to the sketch verifies every proof in the chain and confirms that adjacent proofs agree on their boundary commitment: the output state commitment of π_i must equal the input state commitment of π_{i+1}. Provided the chain is intact and every π_i verifies, the verifier learns that the final committed state is the result of applying all committed updates in order.

The cost of this scheme falls on the verifier. After E updates have been processed, the verifier has to check E proofs and verify $E-1$ boundary equalities between consecutive proofs. Verification time scales as $\Theta(E)$ in the number of updates. For sketches maintained over long data streams, the proof chain grows with the stream length, eventually exceeding what a bounded verifier can reasonably ingest and verify end-to-end.

A natural attempt to escape this bottleneck is to enlarge each update so that fewer of them are needed. Bounded prover memory and verifier compute prevent this: a single ZKP over a very large update becomes prohibitively expensive to generate and to verify, and the deployment's per-update latency budget caps the update size in practice. The per-update strategy therefore cannot escape the $\Theta(E)$ verifier cost simply by growing each update. Local soundness of individual transition proofs is useful, but it is not enough for long streams; Section 4.2 collapses the chain into a single root proof via recursive composition.

4.2 Collapsing the chain via recursive composition

Why we need a stronger primitive. The per-update chain of Section 4.1 leaves the verifier with $\Theta(E)$ proofs to check. The chain links those proofs only structurally — boundary-state commitments tie each π_i to its neighbours, but no proof in the chain ever *verifies* any of the others, so checking one proof tells the verifier nothing

about the rest. What we need is a primitive that lets a single most-recent proof subsume all earlier proofs, so the verifier checks one object regardless of how long the stream is.

Recursive proof composition, for database readers. A zero-knowledge proof system (§2.1) reduces a computation to an arithmetic circuit and emits a short certificate that the circuit accepted its inputs. The system's *verifier* is itself a deterministic algorithm; nothing stops us from encoding that verifier as a circuit and asking a fresh proof to certify a computation that *contains* the verifier as a subroutine. Recursive proof composition [13, 32] does exactly that: a new circuit runs its native computation *and* runs the ZKP verifier on a previously emitted proof, then emits a single new proof that certifies both at once. The new proof is a transitive certificate for everything the old proof certified, plus the new local step.

The database analogue is an incrementally maintained materialized view. Each maintenance step normally carries a local *provenance certificate* attesting "this step is correct given the previous view state." Plain provenance is local: it does not retroactively certify earlier updates, so an auditor wanting to trust the view today still has to check the provenance certificate of every past update. With recursive composition, each new provenance certificate *also* verifies the previous certificate inside the same circuit, so the most-recent certificate transitively certifies the entire maintenance history — an auditor inspects exactly one certificate to be convinced of the whole view. Applied to the per-update chain of Section 4.1: the prover emits a single $\Pi^{(E)}$ certifying that all E transitions were sound, in order, against the public stream of update commitments $C_{b_1}, \ldots, C_{b_E}$. We build the construction step by step.

The merge atom. The basic operation is a *recursive merge step* on two adjacent interval proofs $\Pi_{[i,m]}$ and $\Pi_{[m+1,j]}$ (Figure 2). The merge circuit verifies both children in-circuit, enforces that the shared boundary commitment $\mathsf{Com}(S_m)$ exposed as the left child's output matches the one exposed as the right child's input, combines the two ordered-update digests into one, and emits a single parent proof $\Pi_{[i,j]}$ on the outer boundary $(\mathsf{Com}(S_{i-1}), \mathsf{Com}(S_j))$. The intermediate $\mathsf{Com}(S_m)$ is absorbed: it no longer appears on the parent's public boundary. A verifier holding $\Pi_{[i,j]}$ checks one ZKP and learns that both children were sound *and* agreed on their join point.

The interval-proof contract. We make the construction precise by giving the merge atom a contract. For an interval $[i, j]$ of updates, an interval proof $\Pi_{[i,j]}$ certifies that the committed state at the end of the interval is reachable from the committed state before the interval:

$$\mathsf{Com}(S_{i-1}) \xrightarrow{b_i, b_{i+1}, \ldots, b_j} \mathsf{Com}(S_j).$$

Its public boundary is $\mathsf{pub}_{[i,j]} = (i,\ j,\ \mathsf{Com}(S_{i-1}),\ \mathsf{Com}(S_j),\ H_{[i,j]},\ \mathsf{M})$, where $H_{[i,j]}$ is a collision-resistant digest of the ordered update commitments $C_{b_i}, \ldots, C_{b_j}$ that binds membership and order: omitting, replacing, or reordering an update changes the public input. Two rules complete the contract. A leaf interval is a single update:

$$\Pi_{[j,j]} = \pi_j, \qquad H_{[j,j]} = \mathsf{H}(j,\ C_{b_j}).$$

A merge over any split $[i, m] \cup [m{+}1, j]$ verifies both children against their own public boundaries, checks $\mathsf{Com}(S_m)$ agrees, and combines digests as

$$H_{[i,j]} = \mathsf{H}(i,\ m,\ j,\ H_{[i,m]},\ H_{[m+1,j]}).$$

The verification target on the parent is exactly: every child verifies and every shared boundary agrees.

Running example (part I).. We thread a single running example through this subsection and Section 4.3 that grows with the stream. Suppose the first two updates b_1, b_2 have been processed, with per-update proofs π_1, π_2 binding $\mathsf{Com}(S_0) \xrightarrow{b_1} \mathsf{Com}(S_1)$ and $\mathsf{Com}(S_1) \xrightarrow{b_2} \mathsf{Com}(S_2)$. Lifting each π_j to a leaf interval gives $\Pi_{[1,1]}, \Pi_{[2,2]}$. One application of the merge atom verifies both leaves, checks that the shared $\mathsf{Com}(S_1)$ agrees, combines digests as $H_{[1,2]} = \mathsf{H}(1, 1, 2, H_{[1,1]}, H_{[2,2]})$, and emits

$$\Pi_{[1,2]} : \quad \big(1,\ 2,\ \mathsf{Com}(S_0),\ \mathsf{Com}(S_2),\ H_{[1,2]},\ \mathsf{M}\big).$$

The intermediate $\mathsf{Com}(S_1)$ has been absorbed; the parent exposes only the outer boundary. We continue this example in Section 4.3 as more updates arrive.

Figure placeholder. One recursive merge step (the atom of the construction). *Inputs (bottom of the figure):* two child interval proofs. The left child $\Pi_{\text{left}} = \Pi_{[i,m]}$ carries the public boundary $(\mathsf{Com}(S_{i-1}),\ \mathsf{Com}(S_m),\ H_{[i,m]})$; the right child $\Pi_{\text{right}} = \Pi_{[m+1,j]}$ carries $(\mathsf{Com}(S_m),\ \mathsf{Com}(S_j),\ H_{[m+1,j]})$. Draw each child as a small box with three labelled tabs on its public boundary, and use blue for state-commitment boundaries that will survive into the parent, grey for the shared $\mathsf{Com}(S_m)$ that will be absorbed, orange for the ordered update digests H. *Centre of the figure:* the recursive merge circuit, drawn as a larger box with a small ZKP-verifier icon inside. Internally the box performs three operations: (i) it runs the ZKP verifier on each child against its own public boundary; (ii) it checks that the shared $\mathsf{Com}(S_m)$ exposed as the left child's *output* equals the one exposed as the right child's *input*, highlighted with a dashed equality bracket between the two grey tabs; (iii) it combines the two ordered update digests via $H_{[i,j]} = \mathsf{H}(i, m, j, H_{[i,m]}, H_{[m+1,j]})$, shown as an orange merge node. *Output (top of the figure):* one parent interval proof $\Pi_{\text{parent}} = \Pi_{[i,j]}$ exposing only the outer boundary $(\mathsf{Com}(S_{i-1}),\ \mathsf{Com}(S_j),\ H_{[i,j]})$; the shared $\mathsf{Com}(S_m)$ does not appear on the parent's public boundary (it has been "hidden" behind the merge). *Caption hook:* include a small inset diagram on the right suggesting that iterating this step over a forest of size-2^k subtrees yields a single root proof on demand (forward-pointer to Figure 4).

Figure 2: The atom of recursive composition. Two child interval proofs go in, one parent interval proof comes out. The merge circuit runs the ZKP verifier on each child inside a new circuit, enforces that the shared boundary state commitment $\mathsf{Com}(S_m)$ agrees on both sides, and combines the ordered update digests. Iterating this step over a forest of size-2^k subtrees (Figure 4) collapses an arbitrary-length committed history into one root proof checkable in $O(1)$ time.

The chain structure: a natural single-thread organisation. The simplest way to apply the merge atom to a streaming history is to maintain one accumulating root proof $\Pi^{(t)}$, extending it by one merge each time a fresh leaf arrives: $\Pi^{(1)} = \pi_1$, and at $t \geq 2$ the prover merges $\Pi^{(t-1)}$ with $\Pi_{[t,t]} = \pi_t$ to obtain $\Pi^{(t)}$ (Figure 3a). The prover therefore always holds a current root proof; a query arriving at any moment reads $\Pi^{(t)}$ directly, so the query latency is $O(1)$ *by construction*. The online cost per update is one merge.

Folding-style realisations [32] keep that per-update cost constant in practice: instead of running the full in-circuit ZKP verifier at every step, the prover folds the new instance into an accumulator at constant cost and emits a succinct SNARK only when a verifier asks for one — the recursion depth no longer grows with the stream length. The chain has *one* structural limitation, however: each merge depends on the previous one finishing, so the schedule is purely sequential. A multi-core or GPU prover has no way to use its parallel resources across the stream, and on high-velocity streams this becomes the bottleneck.

Root interface. For a stream prefix of length t, the prover publishes the pair

$$\big(R^{(t)},\, \Pi^{(t)}\big), \qquad R^{(t)} = \mathsf{H}\big(1,\, t,\, \mathsf{Com}(S_0),\, \mathsf{Com}(S_t),\, H_{[1,t]},\, \mathsf{M}\big),$$

where $\Pi^{(t)} = \Pi_{[1,t]}$ is the root interval proof. The verifier checks one succinct proof against the $O(1)$-size root commitment, regardless of t. How the prover can *parallelise* this work to keep up with high-velocity streams, trading some of the chain's $O(1)$ query latency for parallel proving, is the subject of Section 4.3.

4.3 Maintaining the root proof over a streaming history

Section 4.2 introduced the chain organisation of the merge atom: one merge per update, root always ready, $O(1)$ query latency. The chain is sequential by construction — each merge waits for the previous to finish — so on a multi-core or GPU prover it leaves parallel resources idle. High-velocity streams (financial tick data, blockchain analytics, IoT telemetry) routinely produce thousands of updates per second; a chain paying even tens of milliseconds per merge saturates well below line rate. This section asks how to schedule the merges so disjoint pieces of the history are proved concurrently. Two refinements answer that question and differ in how they treat a fixed window versus a live stream; both trade some of the chain's query latency for parallel proving work.

First parallel schedule: the balanced binary tree. When E is known in advance and a power of two, the natural parallelisation organises the E leaf proofs as a balanced binary tree of depth $\lceil \log_2 E \rceil$ (Figure 3b). Every merge in the tree applies the same fixed-shape circuit; disjoint subtrees can be built in parallel across cores; the verifier still checks one root proof. The trade-off versus the chain is concrete: appending a new leaf regenerates the $\lceil \log_2 E \rceil$ merge proofs along the spine from the new leaf to the root, so the per-update latency to refresh the root is $\Theta(\log E)$ rather than $O(1)$. *Query latency is higher than the chain's* when the prover must refresh the root on demand, but throughput on a multi-core prover is far higher. This makes the binary tree the natural choice for windowed batch processing where the prover commits one tree per window.

Figure placeholder. Three schedules for composing interval proofs over a streaming history. (a) *Linear chain*: a single accumulating interval $\Pi_{[1,t]}$ is extended by one merge per arrival; recursion depth of the root proof grows linearly with t. (b) *Balanced binary tree* (idealised, $E = 8$): leaves $\Pi_{[1,1]}, \ldots, \Pi_{[8,8]}$ at the bottom, internal merges at intervals $[1,2]$, $[3,4]$, $[5,6]$, $[7,8]$, $[1,4]$, $[5,8]$, $[1,8]$; requires E known in advance and a power of two. (c) *Streaming forest* (general t, illustrated at $t = 11 = 8 + 2 + 1$): three surviving subtree roots $\Pi_{[1,8]}, \Pi_{[9,10]}, \Pi_{[11,11]}$ of sizes 8, 2, 1. Draw the on-demand chaining as dashed merges from left to right that produce a single root $\Pi^{(11)}$ when a query or verifier request triggers finalisation. All three publish the same verifier interface $(R^{(t)}, \Pi^{(t)})$.

Figure 3: Three composition schedules for interval proofs. Panel (a) is the chain of §4.2 reproduced for comparison; panels (b) and (c) are the parallel schedules developed in this section. The chain is sequential with $O(1)$ query latency; the balanced binary tree is parallelisable but assumes a fixed, power-of-two history; the streaming forest accommodates any t and finalises a single root proof on demand.

The streaming reality. The balanced binary tree assumes E is known in advance and is a power of two. In live streaming neither holds: E is unknown and queries arrive at unpredictable moments. We need a structure that (i) accommodates any current length t, (ii) admits cheap incremental extension as new leaves arrive, and (iii) can deliver a single root proof at any moment on demand.

Second parallel schedule: a forest of perfect subtrees. We organise the current history $[1, t]$ as a forest of perfect interval-proof subtrees whose sizes are the powers of two appearing in the binary representation of t. Writing $t = 2^{k_1} + 2^{k_2} + \cdots + 2^{k_\ell}$ with $k_1 > k_2 > \cdots > k_\ell \geq 0$, the forest holds $\ell \leq \lceil \log_2(t{+}1) \rceil$ surviving subtree roots, one of size 2^{k_r} for each set bit. For instance at $t = 11 = 8 + 2 + 1$ the forest holds three roots $\Pi_{[1,8]}, \Pi_{[9,10]}, \Pi_{[11,11]}$ (Figure 3c). The skeleton is the Bentley-Saxe static-to-dynamic transformation [4], and the layout is structurally analogous to a Merkle Mountain Range [35]; we borrow only the organisation that schedules the recursive merges, and do not derive membership proofs from the forest.

Incremental extension. When π_{t+1} arrives, the prover inserts a fresh size-1 leaf $\Pi_{[t+1,t+1]}$ on the right. If a size-1 subtree was already the rightmost surviving root, the two merge into a size-2 subtree via one application of the merge atom; if a size-2 subtree was also present, the cascade continues, and so on until no two equal-sized neighbours remain. The cascade visits at most $\lceil \log_2(t{+}1) \rceil$ levels and emits $O(\log t)$ merges in the worst case, with $O(1)$ amortised over the stream by the standard binary-counter argument. Merges on disjoint subtrees run in parallel.

Producing the root on demand. At any moment the prover can compose the surviving forest roots $\Pi_{[1,\,2^{k_1}]}, \ldots, \Pi_{[\sum_{r<\ell} 2^{k_r}+1,\,t]}$ into a single root proof $\Pi^{(t)}$ by chaining them left-to-right with

$\ell - 1 < \lceil \log_2(t{+}1) \rceil$ additional merges, and publishes $(R^{(t)}, \Pi^{(t)})$ to the verifier. This finalisation is triggered by a query or by an external verification request; it is not paid on every update. The amortised online cost per update remains $O(1)$, and the on-demand cost of delivering a root is $O(\log t)$ — one logarithmic factor higher than the chain's $O(1)$ ready root, exchanged for the across-stream parallelism the chain could not provide.

Lazy vs. eager peak chain. The on-demand finalisation just described is the *lazy* policy; an alternative *eager* policy maintains a chain over the current peaks at all times, so a query receives a ready-made root proof without further work. Both policies share $O(1)$ amortised total work per insert: lazy concentrates $O(\log t)$ at query time, while eager pays a cascade-aligned chain rebuild at insert time — one extra merge per peak consumed by the cascade. The forest skeleton is Bentley-Saxe's static-to-dynamic transformation [4], and the lazy/eager dichotomy is the standard decomposable-data-structure trade-off, formally analysed in the competitive sense by Mathieu et al. [21]. The closest existing zero-knowledge system, Mina's scan state [23], also maintains a rolling root eagerly, but uses a parallel-scan schedule over full binary trees rather than the peak-chain shape we propose here. Read against the chain of Section 4.2: the eager policy *recovers* the chain's $O(1)$ query latency by paying a chain-rebuild at cascade-aligned insert bursts, so it sits between the pure chain (low query latency, no parallelism) and the lazy forest (parallel, $O(\log t)$ query). zkSketch takes the eager peak chain as its default because streaming workloads issue queries at unpredictable times and the user-visible cost is query latency; the lazy variant remains preferable when queries are rare relative to updates.

Running example (part II).. Continuing the running example: after $t = 2$ a cascade merged two size-1 leaves into the size-2 subtree $\Pi_{[1,2]}$, so the forest is $\{\Pi_{[1,2]}\}$. When π_3 arrives, the new leaf $\Pi_{[3,3]}$ is size-1 and the rightmost surviving root is size-2, so no cascade fires; the forest becomes $\{\Pi_{[1,2]}, \Pi_{[3,3]}\}$. When π_4 arrives, the new leaf $\Pi_{[4,4]}$ merges with $\Pi_{[3,3]}$ into $\Pi_{[3,4]}$, which is itself size-2, so the cascade continues and merges with $\Pi_{[1,2]}$ into $\Pi_{[1,4]}$; the forest collapses to $\{\Pi_{[1,4]}\}$ (Figure 4). Under the lazy policy, on-demand finalisation at any of these moments chains the surviving roots into a single $\Pi^{(t)}$ that the verifier checks in $O(1)$ time against $R^{(t)}$; under the eager policy, the same chain is already maintained, and the cascade at t=4 rebuilds it over the new single peak.

Design-space summary. The three composition schedules trace one curve trading parallel proving work against query latency:

- *Chain* (§4.2): no parallelism across the stream, $O(1)$ query latency at every t. Best for moderate-rate streams or query-dominated SLAs on a single core.
- *Balanced binary tree*: full parallelism across cores, $\Theta(\log E)$ query latency, requires E known and a power of two. Best for windowed audits with regular window boundaries.
- *Streaming forest with eager peak chain*: full parallelism, $O(1)$ query latency under the eager policy (and $O(\log t)$ under the lazy policy), accommodates any t and online queries. The default for zkSketch.

Figure placeholder. Streaming forest after $t = 1, 2, 3, 4$ committed updates. *Panel t=1:* the forest holds a single size-1 leaf $\Pi_{[1,1]}$. *Panel t=2:* the new leaf $\Pi_{[2,2]}$ cascades with the existing size-1 root; the forest collapses to one size-2 subtree $\Pi_{[1,2]}$, with the absorbed $\mathrm{Com}(S_1)$ shown crossed out behind the merge. *Panel t=3:* the new leaf $\Pi_{[3,3]}$ is size-1 and meets a size-2 neighbour; no cascade fires. Two surviving roots $\Pi_{[1,2]}, \Pi_{[3,3]}$. *Panel t=4:* the new leaf $\Pi_{[4,4]}$ cascades twice: first with $\Pi_{[3,3]}$ into $\Pi_{[3,4]}$, then with $\Pi_{[1,2]}$ into $\Pi_{[1,4]}$. The forest collapses to a single size-4 subtree. Dashed arrows above each panel indicate the on-demand finalisation that chains surviving roots into one $\Pi^{(t)}$. Use orange for ordered-digest combinations and grey for absorbed boundary commitments.

Figure 4: Streaming forest evolving with the running example. Each panel shows the forest after one more update arrives; cascades fire whenever adjacent subtrees have equal size. At any time, on-demand finalisation (dashed) produces a single root proof $\Pi^{(t)}$ verifiable in $O(1)$ time. Under the eager peak-chain policy (§4.3), the dashed chain in each panel is what the prover maintains continuously, with the cascade at t=4 rebuilding the chain over the new single peak.

4.4 Answering queries during streaming

Queries arrive interleaved with updates and must observe a clean prefix of the stream. The streaming forest of Section 4.3 supports this directly.

When a query arrives at logical time t, the prover first treats t as a version boundary. Any records that have arrived since the last committed update $\mathrm{Com}(S_r)$ but have not yet been folded into an update event are sealed as a new update b_{r+1}; the prover emits $\mathrm{Com}(S_{r+1})$ and the corresponding per-update proof π_{r+1}, and inserts the new leaf into the streaming forest via the cascade of Section 4.3.

The prover then triggers on-demand finalisation: chain the surviving forest roots into a single root proof $\Pi^{(t)}$ against the root commitment $R^{(t)}$. This pair certifies that $\mathrm{Com}(S_t)$ is reachable from $\mathrm{Com}(S_0)$ by applying the committed updates $b_1, \ldots, b_t$ in order.

For the query itself, the prover emits an answer *ans* together with a *decode proof* π_q certifying

$$ans = A_\tau.\mathrm{Decode}_q(S_t),$$

where the public input of π_q binds to the $\mathrm{Com}(S_t)$ that appears on the public boundary of $\Pi^{(t)}$. The pair $(\Pi^{(t)}, \pi_q)$ therefore ties the answer to exactly the prefix of committed updates visible at time t.

5 Circuit Designs for Sketches

§4 fixed the proof-orchestration layer: per-epoch transition proofs as the basic proving unit, recursive history composition for $O(1)$-cost verification. It deferred the question this section answers: *how is one per-epoch transition proof constructed?* That is, given a committed epoch B_t, an old committed state $\mathrm{Com}(S_{t-1})$, and the published new state $\mathrm{Com}(S_t)$, what does the circuit certify and how? We build the answer step by step.

The strawman: replay the plaintext update inside the circuit. The most direct realisation is to encode the plaintext sketch update record-by-record. For each $u_i \in B_t$ the circuit (a) opens u_i from C_{B_t}, (b) evaluates the keyed hashes $\{h_g(u_i)\}_{g\in[d]}$ in-circuit, (c) opens the touched cells of the current intermediate state S_{i-1} from a commitment $\mathsf{Com}(S_{i-1})$, (d) updates those cells per the sketch's algorithm to produce S_i, and (e) rebinds the state commitment to $\mathsf{Com}(S_i)$. After $m = |B_t|$ such steps the final state S_m is checked against the published $\mathsf{Com}(S_t)$.

Why the strawman is unaffordable. The strawman pays in three structural ways. First, the *witness must materialise all* $m{+}1$ *intermediate sketch states* $S_0, S_1, \ldots, S_m$, because each per-record step depends on the previous one. For a Count-Min state of $|S| = dw \approx 4.6 \times 10^5$ cells (the running parameters $d = 7$, $w = 2^{16}$) and a batch of $m = 1024$, this is $\sim 4.7 \times 10^8$ field elements in the witness for a *single* epoch proof — an order of magnitude beyond the largest practical PLONKish circuit. Second, the m intermediate state commitments $\mathsf{Com}(S_0), \ldots, \mathsf{Com}(S_{m-1})$ must each be opened and rebound in-circuit, paying $\Theta(m)$ commitment-update work plus per-cell openings. Third, the keyed hash evaluations live inside the circuit per record, contributing $\Theta(mdH)$ constraints under any in-circuit hash family (Appendix B quantifies the contribution).

Why this is a different bottleneck from §4.1. §4.1 treated each update as a single proving unit and addressed the verifier-side cost of a long chain of per-update proofs. Each π_i was a black box there. The strawman above instead opens that black box: a single update is internally a group of m records that the prover has accumulated, and the strawman replays the plaintext sketch update record-by-record *inside* that update's circuit, paying its costs m times in the single proof's witness. §5 must therefore answer a question §4.1 did not address: *how are the m records that make up one update handled inside that update's circuit?* The strawman handles them sequentially, paying for m intermediate states; the rest of this section avoids that.

Our approach: certify the net effect, not the procedure. The observation that closes the gap is that the verifier does not care about $S_1, \ldots, S_{m-1}$; the verifier only cares that the published $\mathsf{Com}(S_t)$ is the result of applying $A_\tau.\mathsf{UpdateEpoch}(S_{t-1}, B_t)$. The circuit can therefore *certify the net effect* of B_t on the committed state without materialising any intermediate state: the prover supplies a compact *delta certificate* summarising how the cells of S_{t-1} map to the cells of S_t under B_t, and the circuit verifies the certificate is consistent with C_{B_t}, $\mathsf{Com}(S_{t-1})$, and $\mathsf{Com}(S_t)$. The witness shrinks from $\Theta(m|S|)$ to $\Theta(md + T)$ — the size of the certificate (md generated rows plus T touched cells) rather than the intermediate-state chain.

The rest of §5 develops this idea. §5.1 states the abstract delta-certified maintenance relation $\mathcal{R}^{\tau}_{\mathsf{upd}}$ that every sketch family's per-batch circuit must satisfy; §5.2 packages the per-family choices as a seven-slot interface Π_τ; §5.3 shows the algebra of one slot forces every supported family into one of two structural patterns; §5.4 and §5.5 walk Pattern A (Count-Min) and Pattern B (Greenwald–Khanna) end-to-end at the gate level, with §5.4 closing on three circuit-level optimisations; §5.6 states coverage and limits. HyperLogLog and SpaceSaving follow the same template and appear in Appendices F–G.

5.1 General Delta-Certified Maintenance

Reading "delta certificate" broadly. The name carries the connotation of an additive delta vector, which is exactly the shape used for Pattern A linear sketches (Count-Min) in §5.3. For Pattern A max (HyperLogLog) and Pattern B (SpaceSaving, Greenwald–Khanna), the "delta" is interpreted more broadly as the *per-batch transition specification*: a register-wise max-update for HLL, a top-s retention claim for SpaceSaving, a rank-interval fusion for GK. In all four cases the certificate is bounded, depends only on public capacities, and is verified by low-degree primitives. We keep "Delta-Certified" as the umbrella term for the entire pipeline and read it broadly throughout.

From strawman to abstract relation. The strawman of §5's opener motivates a delta certificate; this subsection states what the certificate must satisfy abstractly, treating per-family choices as parameters. The per-batch update is a sketch-agnostic pipeline. The six tables $B, D, D^{\uparrow}, A, O, N$ of the Count-Min walkthrough in §5.4 are instances of abstract types $\Sigma_B, \Sigma_\Delta, \Sigma_{\Delta\uparrow}, \Sigma_A, \Sigma_O, \Sigma_N$. The cell key $K = (g, i, b)$ for Count-Min is the row-key of Σ_Δ for a general sketch. The segment aggregation over deltas hitting the same cell is the slot $\oplus$. The patch $C_{new} = C_{old} + \mathsf{Com}(\Delta)$ is one realisation of the slot Patch; whole-state replacement is another. §5.2 packages the slots as the seven-slot interface Π_τ and §5.3 shows the two patterns the algebra of $\oplus$ forces on every sketch family we support.

The abstract update relation. The pipeline verifies, for every sketch type τ,

$$\mathcal{R}^{\tau}_{\mathsf{upd}} \;=\; \big\{\, (x_t, w_t) \;:\; S_t = A_\tau.\mathsf{Update}(S_{t-1}, B_t) \,\big\},$$

with public input $x_t = (C_{S_{t-1}}, C_{B_t}, C_{S_t}, \mathsf{M})$ and private witness w_t the six tables of the certificate. The relation is parametric in τ: instantiated for Count-Min via the slot assignment of §5.3, it gives the Count-Min update circuit of §5.4; instantiated for a different sketch family, it gives the corresponding pipeline. Algorithm 1 states this generically. The history-binding requirement — that the sequence of accepted π_t certify a single coherent history — is delegated to the sketch-merge segment tree of §4.3, which appends each new π_t as a leaf and publishes an updated root commitment $R^{(t)}$.

C4 low-degree constraints. Every gate in the pipeline must reduce to low-degree polynomial identities and lookups, the operations PLONKish backends prove cheaply. Comparisons become range checks on differences (one Cmp per row), equality becomes an inverse witness (one Mul plus one Eq), tuple equality becomes a challenge-compressed permutation argument (one challenge plus one Eq per row), and conditionals become linear Mux gadgets (one Mul plus one Add per row). We highlight specific instantiations in Appendix E; the design rule is uniform across families.

Soundness. The update circuit is sound in the following sense.

Theorem 1 (Update soundness, informal). *Assume PLONKish knowledge soundness, computational binding of the commitment scheme, and collision resistance of* H. *For every sketch type τ, the update circuit is knowledge-sound for the relation $\mathcal{R}^{\tau}_{\mathsf{upd}}$ above.*

The proof (Appendix J) follows the standard extract-then-bind pattern. The PLONKish extractor recovers the witness; commitment binding pins the openings of C_{B_t}, C_{old}, and C_{new}. The pipeline steps then force $S' = A_\tau.\mathsf{Update}(S, B_t)$ cell by cell. Multi-batch history

Algorithm 1: Delta-certified update circuit (one batch, sketch-agnostic)

Input: Public $C_{B_t}, C_{old}, C_{new}, \mathsf{M}$; private tables $B, D, D^{\uparrow}, A, O, N$
Output: Proof that C_{new} is the result of applying B_t to C_{old} under A_τ

1 Verify B opens from C_{B_t} and inactive rows are canonical;
2 Generate D from B using sketch-specific Generate gates;
3 Verify $D^{\uparrow}$ is a permutation of D via Permute;
4 Verify LexSort$(D^{\uparrow}, K)$ for sketch key K;
5 Run Segment and SegSum/SegMax to obtain aggregate A;
6 Verify O, N openings against C_{old}, C_{new} at keys in A;
7 Run SparseAddPatch / SparseMaxPatch / ReplaceState (per pattern);
8 Run Compat (history binding via MergeTree-Append, §4.3);

binding follows separately from the segment-tree root composition theorem of §4.3.

Two segment-aggregation realisations, plus one structural alternative. The aggregate $A[\kappa] = \bigoplus_{j:K_j=\kappa} v_j$ admits two PLONK-ish realisations between which the planner chooses (Appendix D). The *local recurrence* witnesses boundary bits $b_j = [K_j \neq K_{j-1}]$, running aggregates r_j, and end bits $e_j = [K_j \neq K_{j+1}]$, with constraints $r_j = b_j v_j + (1-b_j)(r_{j-1} \oplus v_j)$ and A obtained by compacting the rows with $e_j = 1$. The *prefix-difference* realisation witnesses a global prefix sum $p_j = \sum_{r \le j} v_r$ and reads each segment aggregate as a difference $\Delta_{seg} = p_{\text{end}} - p_{\text{before-start}}$, mirroring the optimised primitive used by S^3 [40]. The local recurrence is easier to debug; the prefix-difference variant has smaller constants for long segments. A third option bypasses segment aggregation entirely by routing per-record contributions to per-cell aggregates through a multiset-equality (LogUp) bus; the Count-Min instantiation appears in §5.4.

5.2 The Seven-Slot Interface

The abstract relation $\mathcal{R}^{\tau}_{\text{upd}}$ of §5.1 is parametric in the sketch type τ. We instantiate it for a new family by filling a seven-slot interface tuple — analogous to registering a new aggregate type in a database engine. The operator supplies

$$\Pi_\tau = (\Sigma_S,\ \Sigma_\Delta,\ \oplus,\ \mathsf{Patch},\ \mathsf{Decode},\ \mathsf{Merge},\ \mathsf{Compat}),$$

where Σ_S specifies the committed state schema, Σ_Δ specifies the per-record certificate row schema, $\oplus$ is the aggregation operator that combines contributions to the same cell or shard, Patch rebinds the state commitment from $\mathsf{Com}(S_{t-1})$ to $\mathsf{Com}(S_t)$ given the aggregate A, Decode answers a query against the committed state, Merge combines two compatible committed states, and Compat is a public predicate fixing parameters on which two states must agree to be merge-compatible. The framework provides everything else — correctness checking, planning, serialisation — so adding a new sketch is just filling the seven slots. Formal slot definitions and per-slot realisations appear in Appendix D.

5.3 Two-Pattern Dichotomy

Every supported family is delta-certified in the sense of §5.1, but the *shape* of the delta differs by pattern: a literal additive delta vector for Pattern A linear, a register-wise max-update for Pattern A max, and a structured whole-state replacement under an invariant check for Pattern B. The seven-slot interface is general, but in practice the algebra of the aggregation operator $\oplus$ forces every sketch family we support into one of these two structural patterns. The pattern determines the shape of the certificate and the cost of the patch.

Pattern A: per-cell delta certificate. When $\oplus$ is associative and commutative and the state is a flat array of cells indexed by hash address, the delta certificate of §5.4 applies almost directly: Σ_S is a key- or hash-indexed cell array, Σ_Δ rows target cell ids, and Patch rebinds only the touched cells. Pattern A splits into two sub-regimes by whether $\oplus$ is also *additive*.

When $\oplus$ is additive (Count-Min), the state commitment can be a homomorphic-additive vector commitment, and the patch reduces to a single group equation $\mathsf{Com}(S_t) = \mathsf{Com}(S_{t-1}) + \mathsf{Com}(\Delta)$. The circuit therefore needs no per-cell *Merkle path*: the prover supplies the aggregate table A of T touched cells, the circuit verifies one MSM-consistency check tying $\mathsf{Com}(\Delta) = \sum_{(\kappa,\Delta_\kappa)\in A} \Delta_\kappa \cdot G_\kappa$ to A (cost $\Theta(T)$ scalar multiplications), and a single elliptic-curve equation closes the patch. We call this *Pattern A linear*.

When $\oplus$ is associative-commutative but not additive — HyperLogLog's max is the canonical example — the homomorphic shortcut fails. The patch is then verified by a batched IPA/KZG multi-opening over the T touched cells, plus in-circuit max gates. We call this *Pattern A max*.

Pattern B: mergeable-summary batch certificate. When the state is a structured committed object (a sorted top-counter table, a rank-interval tuple list) rather than a flat cell array, and $\oplus$ is order-sensitive, the per-cell pattern breaks down. Updates change relative orderings, evictions, and rank intervals — there is no fixed-size sparse patch that captures their effect. Instead, the prover supplies the new state directly, and the circuit certifies that the new state satisfies the sketch's mergeable-summary invariant on the union of the old state and the batch (top-s retention for SpaceSaving, GK rank-interval fusion for Greenwald–Khanna). Patch is whole-state replacement under a bounded-depth invariant check; Σ_Δ rows describe operations on the structure rather than per-cell contributions. We call this *Pattern B*; it inherits its theoretical footing from the mergeable-summaries line [1].

The four families. Table 1 summarises the assignment. Count-Min is Pattern A linear; HyperLogLog is Pattern A max; SpaceSaving and Greenwald–Khanna are Pattern B. We work Count-Min and Greenwald–Khanna in detail below; the other two families fill the same seven slots and are deferred to Appendices F–G.

5.4 Pattern A Worked Example: Count-Min

We now instantiate the delta-certified pipeline of §5.1 for Count-Min. The strawman of §5's opener would have stored all m+1 intermediate Count-Min matrices $S_0, \ldots, S_m$ as private witness, plus m in-circuit state-commitment rebindings and md in-circuit hash evaluations; the construction below replaces all of that with a six-table delta certificate of total size $\Theta(md + T)$, which the circuit verifies

Table 1: Interface instantiations across the four sketch families. Body sketches: CM and GK (worked through below). Appendix sketches: HLL (Appendix F), SS (Appendix G).

Sketch	Pat.	Σ_Δ row	$\oplus$	Decode
CM	A	(row, col, δ_v)	sum	row min
HLL	A	(reg, ρ_v)	max	harmonic-mean
SS	B	(key, δ_c)	top-s promote	top-k walk
GK	B	(value, op)	rank-fusion	rank walk

by low-degree primitives in one pass. We walk the certificate on the running batch of Example 3.1, then give the L2 PLONKish realisation as three gate boxes plus the per-merge gate that the segment tree of §4.3 uses.

Fix one batch of m records over a sketch of depth d, with public touched-cell capacity T (the data-independent upper bound on distinct cells the batch may touch). The prover assembles a witness consisting of six tables, which we introduce one at a time so the role of each is clear before the constraints arrive. Throughout this subsection we trace the running batch of Example 3.1 (five transactions over Count-Min with $d = 2$, $w = 4$).

Table B: opening of the batch. The first table holds the records of the batch as they appear inside the circuit. B has capacity m and is opened from the published commitment C_{B_t}, so every active row corresponds to a real record committed by the data owner. For our running batch, B has five active rows: (ACME, 100), (BETA, 50), (ACME, 20

Table D: per-record per-row delta expansion. A plaintext Count-Min update writes to d cells per record. To make all writes visible to the circuit, we expand B into a generated delta table D of capacity md, whose rows have the form (g, k, v, i, b, δ), where i is the row index inside the sketch, $b = h_i(k)$ is the bucket address for that row, and δ is the contribution. For the running batch with $d = 2$ this produces $5 \times 2 = 10$ rows.

Table $D^\uparrow$: sorted by cell key. Random writes inside the circuit are expensive. To replace them with one grouped patch per cell, we sort D by the cell key $K = (g, i, b)$. The result $D^\uparrow$ is the same multiset of rows as D but arranged so that all writes to the same cell are adjacent. The circuit verifies that $D^\uparrow$ is a permutation of D and that its rows are non-decreasing in K.

Table A: aggregate per touched cell. Within each contiguous run of $D^\uparrow$ rows sharing the same cell key, the contributions are combined by the sketch's aggregation operator $\oplus$ (sum for Count-Min). The result is the aggregate table A of capacity T: one row per distinct touched cell, carrying its key κ and its segment aggregate $\Delta[\kappa]$. For the running batch, A has six active rows; the ten generated rows collapse into six distinct touched cells.

Tables O and N: opened cells before and after the patch. The last two tables open the old and new state at exactly the touched cells. $O[\kappa]$ holds the value of the old state at cell κ and is verified against $\mathsf{Com}(S_{t-1})$; $N[\kappa]$ holds the corresponding value of the new state and is verified against $\mathsf{Com}(S_t)$. Both tables have capacity T.

Padding and obliviousness. Inactive rows in every table carry canonical dummy values gated by an active flag, and the active flag is constrained against the public capacity. The circuit shape therefore depends only on the public capacities (m, d, T) and not on which records the batch actually contains.

Update pipeline. With the six tables defined, the update circuit verifies them in a fixed pipeline of eight steps (Algorithm 1): batch binding, generation, conservation, order, aggregation, patch, tree append (§4.3), and compatibility. We now give the PLONKish gate boxes that realise this pipeline. The state is a committed array $S[g, i, b]$ indexed by group g, row $i \in [d]$, and bucket $b \in [w]$, committed under the homomorphic-additive vector commitment of Table **??**; the L3 pipeline maps into three update/decode gates — CM-Generate, CM-Update, CM-PointQuery — plus a per-merge gate CM-Merge used by internal nodes of the segment tree.

Gate CM-Generate: from records to per-cell deltas. The first gate produces the generated delta table D of §5.4. Per record (k, v) and per row i, it emits one row containing the cell address $b = h_i(k)$ and the contribution $\delta = a \cdot op \cdot v$. The work is mostly per-row arithmetic and a hash-profile lookup; the harder cross-record reasoning — collision handling, sortedness, segment aggregation — is deferred to CM-Update.

Gate CM-Generate. Witness. Batch table $B[j] = (a_j, g_j, k_j, v_j, op_j, t_j)$ for $j \in [m]$, opened from C_{B_t}. Generated-delta table $D[j, i]$ for $j \in [m], i \in [d]$ with columns $(a, g, k, v, i, b, \delta, key, hcert)$, where $key = (g, i, b)$ and $hcert$ is the hash-profile witness. **Output.** Generated table D of public capacity md. **Verification target.** For every active row, $b_{j,i} = h_i(k_j)$ and $\delta_{j,i} = a_j \cdot op_j \cdot v_j$. **Realization. (G1) Batch-row linkage:** the row schedule $r = jd + i$ is enforced by a fixed column storing $i = (r \bmod d)$; no lookup on j is needed. **(G2) Field copy:** $D[r].g = B[j].g$, $D[r].k = B[j].k$, $D[r].v = B[j].v$, $D[r].a = B[j].a$ via copy constraints. **(G3) Hash correctness:** HashProfile(M, i, k_j) outputs b and proves $b = h_i(k_j)$ under the declared profile, in NativeHash, LookupHash, or HashCert mode (§**??**). RangeCheck enforces $b < w$. **(G4) Delta correctness:** $\delta = a \cdot op \cdot v$, computed by Mux on a with RangeCheck on v within the counter bound. For insertion-only deployments, $op = 1$. **(G5) Inactive canonicalization:** $(1 - a)(\delta - \perp_\delta) = 0$ for the dummy delta $\perp_\delta = 0$, and similarly for every other field.

Gate CM-Update: certify the batch's effect on the state. This is the heart of the Count-Min instantiation. Given the generated delta table from CM-Generate, the gate runs the eight-step pipeline of §5.4: it certifies that the sorted delta table is a permutation of the generated table, that segment aggregation is correct, and that the state commitment is rebound consistently. Two design choices give the cost saving: (i) the homomorphic-additive vector commitment lets the patch reduce to a single group equation $\mathsf{Com}(S_t) = \mathsf{Com}(S_{t-1}) + \mathsf{Com}(\Delta)$, avoiding per-cell Merkle paths; (ii) the prover supplies a sorted witness, so the sort is verified rather than computed — linear in the certificate size, not $O(n \log n)$ comparator gates.

Gate CM-Update. Witness. Generated D, sorted $D^\uparrow$, aggregate table $A[\kappa]$ of capacity T with columns $(a, g, i, b, \Delta, start, end, pos)$,

old/new opening tables $O[\kappa], N[\kappa]$ of capacity T, and commitment-update witnesses for $C_{old} \to C_{new}$. **Output.** New state commitment C_{new}. **Verification target.** $S'[g,i,b] = S[g,i,b] + \sum_{j:h_i(k_j)=b,\ g_j=g} op_j v_j \quad \forall (g,i,b)$, i.e., the homomorphic-additive group equation $C_{new} = C_{old} + \mathrm{Com}(\Delta)$ where Δ is the dense vector with $\Delta[g,i,b] = \sum_j op_j v_j$ over matching rows. **Realization.** **(U1) Conservation:** $\mathrm{Permute}(D, D^{\uparrow})$ verifies $D^{\uparrow}$ is a permutation of D, including the active flag, δ, and all semantic columns. **(U2) Order:** $\mathrm{LexSort}(D^{\uparrow}, K = (g,i,b))$ via equality-prefix and less-than bits with inactive rows last (§5.4, canonical encodings). **(U3) Segment:** boundary bits $b_j = [K_j \neq K_{j-1}]$ and end bits $e_j = [K_j \neq K_{j+1}]$. **(U4) Aggregation:** SegSum (or the prefix-difference variant) computes $\Delta[\kappa]$ at each segment-end row; A is the compacted copy of segment-end rows. **(U5) Sparse patch:** for each $\kappa \in A$, OpenCell verifies $O[\kappa] = S[\kappa]$ from C_{old}; SparseAddPatch enforces $N[\kappa] = O[\kappa] + \Delta[\kappa]$ with overflow range check; an MSM-consistency check ties $\mathrm{Com}(\Delta)$ to A (Appendix E), and the group-relation $C_{new} = C_{old} + \mathrm{Com}(\Delta)$ replaces per-cell Merkle paths. **(U6) Tree append:** the per-batch proof π_t is appended as the leaf at position t of the sketch-merge segment tree (§4.3); the new root commitment $R^{(t)}$ binds the updated history. **Common pitfalls.** The Permute check (U1) must include the δ and active-flag columns; otherwise a malicious prover could permute keys while changing values undetected. LexSort (U2) must force inactive rows to the end; otherwise the prover could hide active rows after dummies and skip them in aggregation. The aggregate table A must be tied to segment-end rows via e_j; otherwise the prover could omit some segments. Counter range checks must be over integer semantics, not field semantics, to prevent wraparound through the field modulus.

Gate CM-PointQuery: read an estimate from the committed state. A Count-Min point query opens d counters — one from each row at the appropriate hash address — and returns their minimum. The trick is to certify the minimum without an in-circuit sorting network. We do this with a selector-and-comparison gadget: the prover indicates which of the d counters is the minimum via a one-hot selector, and the circuit certifies (a) that the selector picks one counter, and (b) that the picked counter is no larger than each of the other $d-1$. Together these force the answer to equal the minimum.

Gate CM-PointQuery. Witness. Query key q, group g, row buckets $b_i = h_i(q)$, opened counters $c_i = S[g,i,b_i]$, opening witnesses π_i, selector bits $r_i \in \{0,1\}$, answer ans. **Output.** Public answer ans. **Verification target.** $ans = \min_{i\in[d]} c_i, \qquad c_i = S[g,i,h_i(q)] \quad \forall i$. **Realization. (Q1) Hash and open:** for each i, HashProfile certifies $b_i = h_i(q)$ and OpenCell certifies $c_i = S[g,i,b_i]$ against the latest $\mathrm{Com}(S_t)$. **(Q2) Selector:** $\sum_i r_i = 1$ and $r_i(r_i - 1) = 0$, with $ans = \sum_i r_i c_i$, certifying that ans equals exactly one opened counter. **(Q3) Min:** $ans \le c_i$ via Cmp for every i; together with (Q2), this enforces $ans = \min_i c_i$ without requiring an in-circuit sorting network. The cost is $\Theta(d)$ openings plus $\Theta(d)$ comparisons, matching Table **??**.

Three circuit-level optimisations to the Count-Min pipeline. The baseline CM-Generate and CM-Update circuits above suffice for soundness, but three concrete optimisations cut their cost meaningfully without changing the seven-slot interface (§5.2) or the certificate-shape catalog (§**??**). The first reduces the per-record hash cost under NativeHash mode by exploiting Poseidon2's wide state; the second is an architectural alternative to the sort+segment pipeline (U1)–(U4) that routes per-record contributions to per-cell aggregates via a multiset-equality bus; the third deduplicates the batch off-circuit and certifies the dedup via a multiplicity-weighted multiset-equality argument. The three compose, and the planner of §**??** picks among the resulting layouts on per-deployment constants.

Optimisation 1: wide-output Poseidon2. Problem. The baseline CM-Generate gate (G3) verifies d keyed hash outputs per record. Under NativeHash mode, each output is a separate in-circuit Poseidon2 invocation; for $d = 7$, $m = 1024$ the per-batch hash share is $\sim$ 7,168 permutations and dominates the prover cost line.

Insight. Poseidon2 is a permutation on a wide state of t field elements — typical $t = 16$ over BabyBear or KoalaBear in the Plonky3 instantiation — so one permutation call already produces t pseudo-random outputs. Reading $d \le t - 1$ of them at distinct positions gives d keyed hash outputs at the cost of one permutation invocation.

Construction. Per key k_j, set the input state to $(k_j, 0, 0, \ldots, 0)$, apply one Poseidon2 permutation, and read output positions $0, \ldots, d-1$. Each output is reduced modulo w via a range-check decomposition — a clean bit split when w is a power of two, a small lookup otherwise — and the result is the d cell addresses $\{h_i(k_j)\}_{i\in[d]}$. The wide-output extraction is a sub-mode of the NativeHash branch of HashProfile; no new gate is needed and no new certificate-shape row is added to Table **??**.

Savings. The per-record hash share collapses from d Poseidon2 invocations to one: a $d\times$ reduction on the dominant cost line under NativeHash. For the running parameters ($m = 1024$, $d = 7$) this saves $\sim$ 6,144 permutations per batch.

Caveat: PRF heuristic. The d outputs are deterministic functions of one permutation state, not independent draws from d random oracles. The Count-Min error envelope nominally requires pairwise-independent hashes; under the heuristic that Poseidon2's permutation behaves as a random permutation, distinct output positions give pseudo-random values computationally indistinguishable from independent. This is the same heuristic invoked for LowMC in MPC sketches [40], and the PRF-replacement argument ("if each h_i is a PRF, the same error guarantees hold with negligible change in probability") transfers verbatim. The heuristic affects the asymptotic CM error bound, not ZK soundness.

Composition with HPP. Wide-output Poseidon2 optimises NativeHash. Under LookupHash (§**??**), in-circuit Poseidon is eliminated entirely and the trick is moot; under HashCert it does not apply either. The planner of §**??** activates wide-output exactly when the chosen HashProfile mode is NativeHash.

Optimisation 2: multiset-equality routing as an alternative to sort+segment. Problem. The CM-Update steps (U1)–(U4) verify a sort certificate ($D \mapsto D^{\uparrow}$) and a segment-aggregation pass to produce the per-cell aggregate table A. The pipeline is correct and linear in md, but it materialises the sorted table $D^{\uparrow}$ as an explicit witness, enforces lex-order constraints, and tracks equality-prefix

boundary bits — three column families that the simpler one-bus argument below makes optional.

Insight. The aggregate A is uniquely determined by the multiset of cell keys generated by the batch. The circuit can certify this function directly via a single multiset-equality argument: on the hash side, push each $(g_j, i, h_i(k_j))$ with multiplicity 1; on the cell side, pull each (g, i, c) with multiplicity $\Delta[g, i, c]$. The bus balances iff Δ is the correct per-cell aggregate.

Construction. The prover supplies Δ directly as a sparse table of T touched cells (same shape as A in the baseline). For a Fiat–Shamir challenge α, the LogUp identity is

$$\sum_{j\in[m]} \sum_{i\in[d]} \frac{1}{\alpha - \langle g_j, i, h_i(k_j)\rangle} = \sum_{(g,i,c)\in\Delta} \frac{\Delta[g,i,c]}{\alpha - \langle g,i,c\rangle},$$

where $\langle\cdot\rangle$ is the challenge-compressed encoding of the cell-key tuple. The remaining CM-Update steps — (U5) sparse patch and (U6) tree append — carry over unchanged. The routing variant replaces $\{D^{\uparrow},\ A,\ \text{order bits},\ \text{segment bits}\}$ with one LogUp bus over $\{D, \Delta\}$.

Cost comparison. Both designs are linear in $md + T$. Sort+segment costs $\Theta(md)$ permutation + $\Theta(md)$ sortedness + $\Theta(md)$ segment bits + $\Theta(T)$ aggregate. LogUp routing costs $\Theta(md)$ hash-side entries + $\Theta(T)$ cell-side entries. The LogUp variant has fewer column families and reuses the LogUp primitive Plonky3 already ships; the sort variant is more debuggable and reuses the Permute/LexSort primitives shared with Pattern B. The planner of §**??** picks on per-deployment constants; both are valid realisations of the multiset-equality certificate shape (Table **??**, first row).

Caveat: sparsity of Δ. If the prover under-reports touched cells (lists fewer cell-side entries than the hash side requires), the bus identity fails: the cell side cannot absorb all hash-side multiplicities for keys that map to omitted cells. The touched-cell capacity T caps the certificate from above; the LogUp identity caps it from below. The active-flag-with-canonical-dummy convention used in the baseline carries over.

Optimisation 3: batch deduplication. Problem. On skewed workloads (Zipfian web traffic, hot user IDs, repeated identifiers), one batch contains many duplicate keys. Wide-output Poseidon2 still pays one permutation per *record*; duplicates re-evaluate the same hash and waste work proportional to the dedup ratio.

Insight. The prover can deduplicate the batch off-circuit into unique keys $U = (u_1, \ldots, u_u)$ with multiplicities $\mu = (\mu_1, \ldots, \mu_u)$ satisfying $\sum_i \mu_i = m$, evaluate the hash only on U, and certify the dedup via a multiplicity-weighted multiset-equality argument against the committed batch C_{B_t}.

Construction. A new dedup-consistency constraint (C0) is inserted into CM-Generate between (G2) field copy and (G3) hash correctness:

$$\sum_{j\in[m]} \frac{1}{\alpha - k_j} = \sum_{i\in[u]} \frac{\mu_i}{\alpha - u_i}.$$

Constraint (G3) now runs HashProfile only on U (size u, not m). The downstream multiset-equality argument — whether the baseline aggregation (U4) or the LogUp routing of Optimisation 2 — uses multiplicity-weighted entries on the hash side: each unique key u_i contributes μ_i to the cell-side aggregate Δ.

Savings. On a Zipf-$\alpha = 1$ stream (the AOL clickstream benchmark of [40]) with $m = 16{,}384$, the realised u is typically 2,000–4,000, a 4×–8× reduction in Poseidon2 invocations. Combined with wide-output Poseidon2, the per-batch hash share drops to u permutations (one per unique key, d outputs each). On a uniform stream the dedup pays the (C0) LogUp tax with no Poseidon2 savings, so it is workload-conditional.

The fixed-circuit-shape catch. PLONKish backends bake the public dedup capacity $u_{\max}$ into the circuit at compile time. Three deployment options:

(1) $u_{\max} = m$: compiles cleanly but yields no win; do not use.
(2) $u_{\max} < m$ from expected workload skew: real savings; over-skewed batches (with true $u > u_{\max}$) must fall back by splitting into sub-batches or routing to a wider-circuit variant.
(3) Several compiled variants: $u_{\max} \in \{m/8,\ m/4,\ m/2,\ m\}$, host picks per batch based on the realised dedup ratio. This is the production answer and integrates with the planner of §**??** as an additional circuit-variant axis.

Caveat: dependence on the batch commitment. (C0) is non-vacuous only because C_{B_t} is a public input of the per-batch transition proof (§4.1). A prover with no batch commitment to bind against could declare any K for any chosen U. The framework's batch commitment makes (C0) tight.

Composition. Dedup composes with both hash modes (it reduces the number of records on which the chosen mode operates) and with both update designs (sort+segment uses multiplicity-weighted SegSum; LogUp routing uses multiplicity-weighted LogUp entries). Under wide-output + LogUp routing + dedup, the hash share is one permutation per unique key, and the routing argument runs on $m + u(d+1) + T$ LogUp entries total: $m + u$ for the (C0) consistency bus, ud for the hash-side routing bus, and T for the cell side.

Table 2: Per-batch cost of the Count-Min update circuit under combinations of the three optimisations. m **= batch size,** d **= sketch depth,** T **= touched-cell capacity,** $u \le m$ **= realised distinct keys per batch.**

Realisation	Rows verified	Hash invoca
Baseline (sort+segment, NativeHash)	$\Theta(md + T)$	md
+ wide-output Poseidon2	$\Theta(md + T)$	m
LogUp routing (alt. to sort+segment), NativeHash	$\Theta(md + T)$	md
LogUp routing + wide-output	$\Theta(md + T)$	m
LogUp routing + wide-output + dedup	$\Theta(m + u(d{+}1) + T)$	u

Gate CM-Merge: combine two merged states under cellwise sum. The segment tree of §4.3 requires a per-merge circuit that combines two children's committed Count-Min states into the merged state. For Pattern A linear sketches under the homomorphic-additive vector commitment, this is one EC group equation $\mathrm{Com}(S_v) = \mathrm{Com}(S_l) + \mathrm{Com}(S_r)$, and the merge gate is essentially free.

Gate CM-Merge. Witness. Child commitments $\mathrm{Com}(S_l)$, $\mathrm{Com}(S_r)$; merged commitment $\mathrm{Com}(S_v)$. **Output.** Merged commitment $\mathrm{Com}(S_v)$. **Verification target.** $S_v[g,i,b] = S_l[g,i,b] + S_r[g,i,b]$ for every (g,i,b). **Realization. (M1) Group equation:** $\mathrm{Com}(S_v) =$

$\mathsf{Com}(S_l) + \mathsf{Com}(S_r)$ as one elliptic-curve point addition, exploiting the homomorphic-additive vector commitment of §5.4. **(M2) Range check:** per-cell sum stays within the counter bound under deployment-declared rate limits (or wraps in the explicit modular-arithmetic regime). The cost per internal node is therefore one EC equation plus the recursive Plonky3 verification of the children's Π_l and Π_r — a trivially cheap merge gate.

HLL: Pattern A non-additive sub-regime. HyperLogLog is the Pattern A max representative: registers update by max rather than +, so the homomorphic shortcut available to CM does not apply, and the patch is verified by a single batched IPA/KZG multi-opening over the touched registers. The per-merge gate for HLL is cellwise max via Cmp + Mux, costing $O(|S|)$ constraints per internal node (state-size-proportional). Full row-level realization in Appendix F; the (F2) limit (§??) explains why the in-circuit harmonic-mean estimator cost cannot be eliminated.

5.5 Pattern B Worked Example: Greenwald–Khanna

For Pattern B's representative we work the Greenwald–Khanna (GK) summary [12] for approximate quantiles end to end. The committed state is a sorted tuple table T that summarises the empirical rank distribution of all records seen so far to a rank-error tolerance controlled by the public capacity parameter s. Each row $T[r] = (a, e, g_1, g_2, \Delta_1, \Delta_2, t)$ carries an active flag a, a value e (one observed record value), a pair of rank-gap contributions (g_1, g_2) inherited from left- and right-fused neighbours, a pair of rank uncertainties (Δ_1, Δ_2), and a sequence index t used for deterministic tie-breaking under fusion.[1] The state is committed under a fixed-capacity vector commitment of public size $s'(N) = \lceil 2s \ln(N/s + 2) + 2 \rceil$, where N is the cumulative record count, and the table is kept sorted by e. Updates are order-sensitive: a new record may need to be inserted between two existing tuples and then participate in a wave of fusions with its neighbours.

Why the strawman is even worse here than for Pattern A.. The plaintext GK update for one record binary-searches the insertion position, splices in a new tuple, and then walks the table running a fusion sweep that merges adjacent tuples whose combined rank gap plus rank uncertainty stays at or below the public threshold $\theta = \lfloor N/s \rfloor$. Every step is data-dependent: the insertion index depends on the value, the fusion sweep terminates conditionally, and a single record may trigger zero or many fusions. Inside a circuit this becomes a worst-case-padded scan per record plus a worst-case fusion sweep, $\Theta(m\, s'(N))$ rows per batch with non-uniform shape between batches. The data-dependent control flow would also break circuit obliviousness (§3.5). For Pattern A linear we replaced the strawman with a sorted, segment-aggregated delta vector; for Pattern B we replace it with a *mergeable-summary batch certificate* that supplies the new state directly and proves it satisfies the bounded-GK invariant on the union of the old table and the buffered records.

[1] The double-component encoding $(g_1, g_2, \Delta_1, \Delta_2)$ is the bounded-GK variant of S^3 [40]; ordinary GK collapses each pair into a single value. The double form makes merges across shards soundness-preserving.

The four-gate pipeline. zkSketch avoids the direct encoding entirely. We build the post-batch GK state *outside* the circuit by (i) merging the old tuples and the new records into a single sorted union, (ii) initialising the rank intervals of the inserted rows from their old-table neighbours, (iii) applying $\lceil \log s'(N+m) \rceil$ rounds of pairwise tuple fusion under the threshold θ, and (iv) recommitting the compacted result. The proof decomposes into four gates that match the abstract delta-certificate pipeline of §5.4: GK-InsertSort produces the sorted union (the GK analogue of $D \to D^{\uparrow}$); GK-InitDelta fills the rank-interval columns of the inserted rows (the analogue of producing the aggregate table A); GK-FusionRounds realises Patch as a bounded sequence of pairwise FuseTuple certificates; ReplaceState recommits the compacted state. The pipeline replaces data-dependent control flow with one Permute, one LexSort, a fixed number of fusion rounds, and one whole-state recommit.

Gate GK-InsertSort: bring old tuples and new records into one sorted table. GK-InsertSort merges the active rows of the old state S with the records of the batch B into a unified table U of capacity $s'(N) + m$, sorted ascendingly by e with deterministic tie-breaking by sequence index t. Old rows are copied unchanged; new rows are emitted with $g_1 = g_2 = 0$ and placeholder Δ_1, Δ_2 to be filled by GK-InitDelta.

Gate GK-InsertSort. Witness. Old state table S of capacity $s'(N)$ (opened from $C_{S_{t-1}}$) and batch rows $B[j] = (a_j, e_j, t_j)$ of capacity m (opened from C_{B_t}). Union table U of capacity $s'(N) + m$ with columns $(a, e, g_1, g_2, \Delta_1, \Delta_2, t, role)$ where $role \in \{\text{old}, \text{new}\}$. **Output.** Sorted union table U. **Verification target.** U is a permutation of $S \uplus B$ ordered by (e, t); old rows carry their existing $(g_1, g_2, \Delta_1, \Delta_2, t)$; new rows carry $(g_1, g_2) = (0, 0)$ and placeholder deltas; rows are sorted by (e, t) with inactive rows last. **Realization. (I1) Old copy:** for $r \in [s'(N)]$, copy $S[r]$ into the union with $role = \text{old}$ and active flag $S[r].a$. **(I2) New copy:** for $j \in [m]$, set $(a, e, g_1, g_2, \Delta_1, \Delta_2, t, role) = (B[j].a, B[j].e, 0, 0, \perp_\Delta, \perp_\Delta, B[j].t, \text{new})$. **(I3) Conservation:** $\text{Permute}(S \uplus B, U)$ on the compressed encoding verifies U as a multiset of $S \uplus B$, including active flags and role tag. **(I4) Order:** $\text{LexSort}(U, (e, t))$ with inactive rows last. **(I5) Inactive canonicalization:** dummy rows in U carry canonical zero values.

Gate GK-InitDelta: initialise rank intervals for the inserted rows. After GK-InsertSort, each $role = \text{new}$ row sits between two $role = \text{old}$ neighbours (or at a boundary of the table). GK-InitDelta sets the new rows' rank uncertainties (Δ_1, Δ_2) from their nearest old-role neighbours via a forward prefix-copy and a backward suffix-copy recurrence over U; new rows with no old neighbour on a given side take 0 on that side. The closed-form depends on the bounded-GK rank-interval algebra of S^3 [40] and we treat it as a parameter of the gate. The recurrences are linear in the table size and use only Mux, Copy, and RangeCheck primitives.

Gate GK-InitDelta. Witness. The sorted union U from GK-InsertSort; updated columns $\Delta_1[U], \Delta_2[U]$ for $role = \text{new}$ rows; prefix witness $L[U]$ and suffix witness $R[U]$. **Output.** Initialised union $U^{(\Delta)}$ with Δ_1, Δ_2 populated on every row. **Verification**

target. For every $role$ = new row $U[r]$, $\Delta_1[r]$ equals the bounded-GK left-neighbour function applied to the nearest preceding $role$ = old row (or 0 if none), and $\Delta_2[r]$ equals the symmetric right-neighbour function on the nearest succeeding old row. Old-role rows keep their existing Δ values. **Realization. (D1) Prefix copy:** forward recurrence $L_r = \mathrm{Mux}(U[r].role = \text{old}, \phi_L(U[r]), L_{r-1})$ with $L_{-1} = 0$, where ϕ_L is the bounded-GK left-neighbour function [40]. **(D2) Suffix copy:** backward recurrence $R_r = \mathrm{Mux}(U[r].role = \text{old}, \phi_R(U[r]), R_{r+1})$ with $R_{s'(N)+m} = 0$. **(D3) Apply:** for $role$ = new rows, set $\Delta_1[r] = L_{r-1}$ and $\Delta_2[r] = R_{r+1}$; old rows keep their existing values. **(D4) Range check:** all $g_1, g_2, \Delta_1, \Delta_2$ within the public counter bound.

Worked walk-through on Example 3.1. Take the same five-record batch (values 100, 50, 200, 75, 30) and run the pipeline at $s = 2$ from an empty old state. GK-InsertSort produces the sorted union $U = [(e{=}30), (e{=}50), (e{=}75), (e{=}100), (e{=}200)]$ with all rows $role$ = new, $g_1 = g_2 = 0$, and t derived from the batch position so ties break deterministically. GK-InitDelta finds no old-role neighbours on either side, so every $\Delta_1 = \Delta_2 = 0$. With $N = 5$ and $s = 2$, the threshold is $\theta = \lfloor N/s \rfloor = 2$; the FuseTuple condition $g_1^{(L)} + g_2^{(L)} + g_1^{(R)} + \Delta_1^{(R)} + 1 \le 2$ is satisfied for adjacent pairs of fresh rows, so the first round of GK-FusionRounds fires the left-to-right fusions $(e{=}30) \to (e{=}50)$ and $(e{=}75) \to (e{=}100)$, leaving $U^{(1)} = [(e{=}50, g_1{=}1), (e{=}100, g_1{=}1), (e{=}200)]$. The second round fuses $(e{=}50) \to (e{=}100)$ into $U^{(2)} = [(e{=}100, g_1{=}2), (e{=}200)]$, which fits the capacity $s'(5) \approx 5$ and is recommitted as $\mathrm{Com}(S_t)$. The example illustrates how the prover does the algorithmic work outside the circuit; the circuit checks one Permute, one LexSort, a small fixed number of pairwise FuseTuple certificates, and one whole-state recommit.

Gate GK-FusionRounds: certify the bounded-depth compression. This is the heart of the GK instantiation. The fusion step compresses $U^{(\Delta)}$ by applying $L = \lceil \log s'(N{+}m) \rceil$ rounds of pairwise FuseTuple, alternating between even and odd row-offset parity across rounds. FuseTuple is a local certificate on an adjacent pair $(U[r], U[r{+}1])$ that emits one of three outcomes: keep both rows unchanged, fuse r into $r{+}1$, or fuse $r{+}1$ into r. Fusion fires only when the merged tuple's combined rank-gap plus rank-uncertainty stays at or below the public threshold $\theta = \lfloor N/s \rfloor$ and the sequence-index relation $R_1.t > R_2.t$ (or its symmetric form) holds, keeping fusion deterministic across honest provers.

Gate FuseTuple. Witness. A pair of adjacent rows $R_1 = U[r], R_2 = U[r{+}1]$; direction bits $c_L, c_R \in \{0, 1\}$; output pair R_1', R_2'. **Verification target.** $c_L + c_R \le 1$. If $c_L = 1$ (fuse R_1 into R_2): $R_1.a = R_2.a = 1$, $R_1.t > R_2.t$, $R_1.g_1 + R_1.g_2 + R_2.g_1 + R_2.\Delta_1 + 1 \le \theta$, $R_1' = (\text{inactive}, \perp)$, $R_2' = (1, R_2.e, R_2.g_1 + R_1.g_1 + R_1.g_2 + 1, R_2.g_2, R_2.\Delta_1, R_2.\Delta_2, R_2.t)$. If $c_R = 1$ (fuse R_2 into R_1): symmetric. If $c_L = c_R = 0$: $R_1' = R_1$ and $R_2' = R_2$. **Realization. (F1) Selector:** $c_L(c_L - 1) = 0$, $c_R(c_R - 1) = 0$, $c_L + c_R \le 1$. **(F2) Threshold:** Cmp on $R_1.g_1 + R_1.g_2 + R_2.g_1 + R_2.\Delta_1 + 1 \le \theta$ gated by c_L; symmetric Cmp gated by c_R. **(F3) Sequence:** Cmp on $R_1.t > R_2.t$ gated by c_L; symmetric for c_R. **(F4) Conditional update:** Mux on (c_L, c_R) writes the fused or unchanged row contents into R_1', R_2'. **(F5) Range check:** every output column within the public counter bound.

Gate GK-FusionRounds. Witness. The initialised union $U^{(\Delta)}$ from GK-InitDelta; per-round witnesses $U^{(1)}, \ldots, U^{(L)}$ with $L = \lceil \log s'(N{+}m) \rceil$; compacted state S' of capacity $s'(N{+}m)$. **Output.** Certified new GK state S' committed via ReplaceState into C_{S_t}. **Verification target.** Each $U^{(\ell+1)}$ is the result of applying FuseTuple to every adjacent pair of $U^{(\ell)}$ at the alternating even/odd offset of round ℓ; S' is the StableCompact of $U^{(L)}$ restricted to its active rows. **Realization. (R1) Per-round fusion:** for each round $\ell \in [L]$ at the round's parity offset, apply FuseTuple to every adjacent pair; non-fired rows pass through unchanged. **(R2) Compact:** StableCompact moves active rows of $U^{(L)}$ to the front of S' in their existing order; inactive rows carry canonical dummies. **(R3) Recommit:** ReplaceState binds $\mathrm{Com}(S_t)$ to the contents of S' via one MSM of size $s'(N{+}m)$. *Why this works.* The bounded-GK guarantee [40] states that $L = \lceil \log s'(N{+}m) \rceil$ rounds of pairwise fusion suffice to compress any sorted union of bounded shape down to the GK capacity while preserving the εN rank-error invariant. The certificate therefore asserts a global "valid bounded-GK summary" property by checking a fixed number of local pairwise-fusion certificates — the Pattern B analogue, for ordered structured states, of the threshold-residual certificate used for SpaceSaving's top-s retention (Appendix G).

Gate GK-RankQuery: read a rank or quantile from the committed state. A rank query against the GK state asks for the approximate rank of a query value q within the streamed distribution. The state already records the partial cumulative count at each row via $g_1{+}g_2{+}1$. GK-RankQuery opens the row bracket surrounding q, computes the prefix-sum certificate, and returns the midpoint of the rank interval $[r_{\min}, r_{\max}]$ plus a publicly-bounded noise term r_B from the capacity co-sketch (CCS) auxiliary state.

Gate GK-RankQuery. Witness. Query value q; opened rows $T[i^*], T[i^*{+}1]$; prefix-sum witness $P_i = \sum_{j \le i}(T[j].g_1 + T[j].g_2 + 1)$; selector bits $r_i \in \{0, 1\}$ with $\sum_i r_i = 1$ at $i = i^*$; CCS noise witness r_B; answer ans. **Output.** Public answer ans (approximate rank of q). **Verification target.** $ans = (r_{\min} + r_{\max})/2 + r_B$ where $r_{\min} = P_{i^*} - T[i^*].\Delta_2 - T[i^*].g_2$ and $r_{\max} = P_{i^*} + T[i^*{+}1].\Delta_1 + T[i^*{+}1].g_1$. **Realization. (Q1) Prefix sum:** the witness P is verified by the recurrence $P_i = P_{i-1} + T[i].g_1 + T[i].g_2 + 1$. **(Q2) Bracket:** selector bits enforce $T[i^*].e \le q < T[i^*{+}1].e$ via Cmp on the opened rows, with $\sum_i r_i = 1$ pinning i^*. **(Q3) Answer:** ans is computed from the closed-form above with one Mul and three Adds; the noise term r_B is opened from the CCS auxiliary commitment and range-checked against the public CCS budget.

Gate GK-Merge: combine two GK summaries into one. The segment tree of §4.3 requires a per-merge circuit that combines two children's GK tables. We instantiate it as the asymmetric "insert-right-into-left" merge consistent with the segment tree's left-to-right convention: the right child S_r is treated as a batch of records to be merged into the left child S_l. The merge reuses the four-gate pipeline of this subsection unchanged, with the merged-population threshold $\theta = \lfloor (N_l + N_r)/s \rfloor$ computed from the per-child population counts carried in the summary metadata.

Gate GK-Merge. Witness. Child states S_l, S_r opened from $\mathrm{Com}(S_l)$, $\mathrm{Com}(S_r)$; merged state S_v; merged commitment $\mathrm{Com}(S_v)$; per-child populations N_l, N_r. **Output.** Merged commitment $\mathrm{Com}(S_v)$ committing a bounded-GK summary for the combined population $N_l + N_r$. **Verification target.** S_v is a valid bounded-GK summary of the combined population: every active row satisfies $g_1 + g_2 + \Delta_1 + \Delta_2 \le \theta$ for $\theta = \lfloor (N_l + N_r)/s \rfloor$, and the rank intervals are consistent with the union of S_l and S_r. **Realization. (M1) InsertSort:** run GK-InsertSort with S_l as the "old state" and the active tuples of S_r as the "batch records". **(M2) InitDelta:** run GK-InitDelta against the merged neighbour structure. **(M3) FusionRounds:** run GK-FusionRounds with $\theta = \lfloor (N_l + N_r)/s \rfloor$ across $L_{\text{merge}} = \lceil \log s'(N_l + N_r) \rceil$ rounds. **(M4) Recommit:** ReplaceState binds $\mathrm{Com}(S_v)$ to the contents of S_v. Cost per internal node: $O(s' \log s')$ constraints, the same order as the per-batch update, with no new gate primitives.

Why the four-gate pipeline beats a direct circuit. The naive online GK update would pay $\Theta(m\, s'(N))$ per batch for m binary-search insertions and the conditional fusion sweep, with data-dependent control flow incompatible with both the cost target and circuit obliviousness. The four-gate pipeline instead pays $\Theta(s'(N)+m)$ for InsertSort, $\Theta((s'(N) + m)\log(s'(N) + m))$ for LexSort, $\Theta(s'(N+m))$ per fusion round across $\lceil \log s'(N+m) \rceil$ rounds, and $\Theta(s'(N+m))$ for ReplaceState — total $\Theta(s'(N+m) \log s'(N+m))$ per batch, matching Table **??**. The control flow is fully data-independent: every batch produces a circuit of identical shape conditional on the public capacities (s, s', m). This is the Pattern B insight, instantiated for GK: the prover does the sketch's algorithmic work outside the circuit, and the circuit verifies that the result satisfies the bounded-GK invariant on the union of the old state and the batch [40].

5.6 Coverage and Other Sketches

The remaining two families fill the same seven slots. HyperLogLog (Pattern A max with a leading-run hash) is in Appendix F; Space-Saving (Pattern B with top-s retention via the threshold-residual certificate, in place of GK's rank-interval fusion) is in Appendix G. The framework supports any sketch whose update is expressible as either Pattern A or Pattern B with a constant-depth aggregation gate; counting Bloom variants, MinHash, KLL [15], t-digest, and the broader class of mergeable summaries [1] are direct extensions, with KLL worked through as the extensibility example in §**??**. Out of scope: sketches whose merge invariant is non-constant-depth, online learned summaries with adaptive structure, and sketches whose accuracy guarantee relies on randomness that cannot be committed in advance under HPP.

6 Evaluation (in progress)

This section describes the evaluation plan. Empirical results are not included in this draft; they will appear in the camera-ready version. The analytical evaluation in Appendix B grounds the design choices in constraint-count derivations validated against published Halo2 cost microbenchmarks, and Table **??** reports per-sketch planner outputs under three deployment profiles. A convincing SIGMOD/VLDB evaluation should validate the comparative analysis empirically and quantify each compositional and framework-level mechanism's contribution; we organise the planned experiments around three research questions.

6.1 Research Questions

The evaluation answers three research questions that mirror the contribution structure of §1.

- **RQ1 (Capability).** Does delta-certified maintenance preserve sketch accuracy and produce checkable proofs across all four families (CM, HLL, SS, GK), under representative workloads drawn from the regulated-reporting deployment of Example **??**?
- **RQ2 (Scalability).** How do prover time, verifier time, proof size, and memory scale along each cost dimension — state size s, depth d, batch size m, batch rate b, audit-window length E, coupled-sketch count K — compared to optimised exact-ZK-index and zkVM baselines?
- **RQ3 (Component contribution).** How much does each compositional technique (SDC, HPP, NDC, CCS) and each framework-level mechanism (deployment-aware compiler, workflow decomposition) contribute to the end-to-end cost, measured against the optimised single-sketch baseline and against the `naive-zkSketch` strawman of §**??** as a rhetorical upper bound?

6.2 Experimental Setup

Implementation. The prototype targets a Halo2-style PLONKish backend in Rust, reusing standard gates for range checks, sorting, lookups, equality flags, and arithmetic aggregation, following the methodology of PoneglyphDB and VPJoin [2, 14]. The implementation exposes the slot catalog of Appendix D so the deployment-aware compiler of §**??** can pick layouts per deployment profile.

Implementation roadmap. The implementation is staged in three increments. *Stage 1 (per-sketch update circuits):* implement CM, HLL, SS, GK update gates against a single-sketch baseline; validate against the comparative analysis (RQ1 and the small-s regime of RQ2). *Stage 2 (compositional techniques):* add SDC, HPP, NDC, CCS as planner-controlled extensions; run the per-technique ablation of RQ3. *Stage 3 (planner and workflows):* add the deployment-aware compiler, the three-dimensional cost surface, and workflow decomposition; run the full RQ2 sweep and the RQ3 framework-mechanism ablation. The Halo2 backend choice is the simplest path to a working prototype because PoneglyphDB and ZKSQL provide reference implementations of the relational operator gates we reuse; we do not commit to a specific Halo2 fork (PSE, Axiom, or Scroll).

Reproducibility plan. The implementation, benchmark scripts, datasets (synthetic generators + pointers to public real datasets), and per-experiment configuration files will be released under an open-source license at the time of submission. Evaluation tables will report median and 95%-quantile of five trials; we will include a per-experiment cost-model prediction column to validate the analytical model of §**??** against measured numbers.

Workloads. (i) Frequency / range-frequency over filings/transactions (CM); (ii) distinct counts over high-cardinality keys (HLL); (iii) top-k heavy hitters (SS); (iv) quantile queries on numerical fields (GK); (v)

multi-sketch group-by exercising SDC; (vi) insert-heavy and delete-aware mixes exercising NDC; (vii) release policies (thresholding, rounding, optional noisy release).

Datasets. Synthetic uniform, Zipfian, and Gaussian record streams to control skew and collision behaviour; real datasets matching the deployments (SEC filings or simulated regulatory data; on-chain transactions; network flow traces). Dataset choice exercises both high-cardinality and skewed regimes.

Baselines. (i) Naive exact ZK index (Merkle leaves); (ii) optimised exact ZK index (sorted-index commitment with lookups, in the style of PoneglyphDB and ZKSQL); (iii) optimised sketch (this paper, with all four compositional techniques and the planner enabled); (iv) zkVM baselines (RISC0 and SP1) [31, 34]; (v) plaintext sketch engine (Apache DataSketches [3]) as a lower-bound reference. The `naive-zkSketch` strawman of §**??** appears as a rhetorical reference point in the RQ3 discussion; its asymptotic cost is derived analytically in Appendix B rather than measured, because its $\Theta(mdH \log w)$ per-batch cost would dominate any practical sweep before reaching the regime where the optimised sketch operates. Reporting baselines that run out of memory or time as `OOM`/`TLE` rather than silently skipping them, following the convention of ZK-Value [38], makes the comparison's failure modes explicit.

Metrics. Circuit rows, custom-gate counts, lookup counts, prover time, witness-generation time, verification time, proof size, memory, state size, commitment-opening cost, query latency, batch update throughput, touched-cell count, approximation error. Per technique: SDC sharing factor as K varies; HPP multi-opening cost vs. batch size; NDC overhead vs. deletion fraction; CCS DP budget vs. padding overhead. For the planner: the three-coordinate cost surface (§**??**) instantiated for the three deployment profiles.

6.3 RQ1 — Capability

RQ1 is answered by running each of the four families against workload classes (i)–(iv). For each family the experiment certifies four things: (a) the update circuit accepts on honest witnesses across the full sweep of s, d; (b) the decode circuit reproduces the plaintext approximate answer to within the family's stated error bound; (c) the sketch-merge segment tree of §4.3 commits an audit window of E batches with verifier-side cost matching the predicted $O(1)$ (root proof) or $O(\log E)$ (per-batch maintenance); (d) a tampered batch — 5% row substitution, mirroring the fraud scenario of Example **??** — is rejected by the verifier with overwhelming probability.

Takeaway for RQ1. Expected: zkSketch produces checkable proofs for every workload across the four families, and tampering with 5% of rows in any batch is detected via the commitment-binding chain (requirement R-2 of §1). The approximation error matches the family's analytical bound (a sketch property, not a ZK one).

6.4 RQ2 — Scalability

RQ2 is answered by sweeping each cost dimension while holding the others fixed at the running-example default, mirroring the dimension-by-dimension panels used by zk-Value (Figure 3 of [38]). Specifically: $s \in \{2^{10}, \ldots, 2^{18}\}$ for state size; $d \in \{1, \ldots, 16\}$ for depth; $m \in \{2^6, \ldots, 2^{14}\}$ for batch size; $b \in \{1, \ldots, 10^3\}$ batches per hour for batch rate; $E \in \{1, \ldots, 10^4\}$ for audit-window length; $K \in \{1, 2, 3, 5\}$ for coupled sketches. For each sweep we report prover time, verifier time, proof size, and peak memory; the optimised exact-ZK-index and zkVM baselines provide the comparison.

Takeaway for RQ2. Expected: prover time scales linearly in m and T and sub-linearly in s (touched-cell-only commitment work); verifier time scales linearly in E, or logarithmically with the segment-tree extension; proof size is constant per batch (single Halo2 proof). The per-record update cost ratio against the optimised exact index grows with n in the optimised sketch's favour, validating requirement R-3 of §1.

6.5 RQ3 — Component contribution

RQ3 is answered by ablating each component on the workload it targets, with the optimised single-sketch baseline as the reference and the `naive-zkSketch` strawman of §**??** as the rhetorical upper bound. *SDC*: vary $K \in \{1, 2, 3, 5\}$ on multi-sketch group-by, measuring the sort-share reduction predicted in §**??**. *HPP*: vary m and d on Count-Min with hash-profile sizes $N \in \{2^{16}, 2^{20}\}$, measuring the crossover where committed-lookup pays back its $\Theta(N \cdot H)$ preprocessing. *NDC*: vary deletion fraction $f \in \{0, 0.1, 0.5\}$, validating that signed deltas match additive cost up to the range-check overhead. *CCS*: vary DP scale σ across the static / adaptive / DP-via-CCS policies of Table 12, validating the predicted $\sim$ 1% auxiliary-sketch overhead and the $(\varepsilon_N, \delta_N)$ leakage accounting. *Compiler*: contrast planner-chosen layouts across the three deployment profiles against a fixed-baseline layout. *Workflow decomposition*: vary E and shard count K to validate the flat-DAG composition cost.

We additionally ablate the three Count-Min circuit-level optimisations of §5.4; the same knobs reappear in the appendix instantiations for HLL and SS (Appendices F and G) and reuse the same experimental machinery. *Wide-output Poseidon2* (§5.4): sweep depth $d \in \{2, 4, 7, 16\}$ on the CM update circuit under NativeHash; measure Poseidon2 permutation count per batch with and without the wide-output trick. Predicted reduction from md to m permutations per batch ($d\times$ on the dominant cost line under NativeHash). Re-run on HLL with $d = 2$ outputs per record for the 2× predicted reduction. *LogUp routing* (§5.4): compare the baseline sort+segment pipeline against LogUp routing on the CM update circuit at $m \in \{1024, 16384\}$, $d = 7$; report constraint count, column-family count, and prover wall time. Predicted near-equivalent total cost with ∼30% fewer column families. Re-run on the SS-Group step. *Batch deduplication* (§5.4): sweep Zipf parameter $\alpha \in \{0, 0.5, 1, 1.5\}$ on $m = 16{,}384$, $d = 7$; measure realised distinct-key count u and Poseidon2 invocations with wide-output + dedup vs wide-output alone. Predicted 4×–8× hash-cost reduction at $\alpha = 1$ on the AOL clickstream benchmark of [40]. Re-run on HLL and SS with the family-appropriate cost line.

Takeaway for RQ3. Expected: each compositional technique delivers its predicted asymptotic saving on its target workload; each circuit-level optimisation delivers its predicted hash-count or column-count reduction at its activation conditions; the planner chooses materially different layouts across deployment profiles with measurable per-profile cost gaps; and the gap between the optimised sketch and `naive-zkSketch` approaches the analytical $\sim 80\times$ derived in

§??. Together these confirm that each framework component pays for itself in the regime where it activates.

6.6 RQ4 — Composition topology trade-off

RQ4 validates the design-space summary of §4.3 by measuring per-update cost and query latency under the three composition topologies (linear chain, balanced binary tree, streaming forest) and the two peak-chain policies (lazy vs. eager). The sweeps fix CM as the per-batch update circuit — the topology choice is sketch-agnostic — and vary the topology and policy as orthogonal axes.

4-a. Insert-rate scaling on a single-core prover. Sweep batch rate $b \in \{1, 10, 10^2, 10^3\}$ batches/sec on one core; report per-update online cost for the chain (one merge), the balanced binary tree (the $\Theta(\log E)$ spine refresh at $E = 1024$), and the streaming forest (cascade amortised and worst-case). Expected: the chain wins at low rate, the streaming forest's amortised $O(1)$ wins at high rate; the binary tree sits in between under its known-E caveat.

4-b. Insert-rate scaling on a multi-core prover. Repeat 4-a on a 16-core machine. Expected: the chain stays single-threaded and saturates at the same rate as 4-a; the balanced binary tree and the streaming forest scale to parallelism, validating the parallelisation motivation of §4.3.

4-c. Query latency under lazy vs. eager peak chain. Sweep query rate q from 1/min to 1/sec while holding the insert rate at 10^2 batches/sec; measure query latency (from query arrival to verifier-checked root proof) and P99 insert latency (cascade-aligned bursts). Expected: under lazy, query latency follows $O(\log t)$ and P99 insert latency follows the cascade depth; under eager, query latency is $O(1)$ and P99 insert latency picks up the cascade-rebuild surcharge. The crossover at which eager dominates matches the predicted query-to-insert ratio threshold of §4.3.

Takeaway for RQ4. Expected: the topology and peak-chain policy each have a measurable crossover point on the batch-rate / query-rate plane, and the policy default proposed in §4.3 (streaming forest with eager peak chain) sits on the Pareto frontier for streaming SLAs with unpredictable query arrival.

Table 3 maps each RQ-level claim to the experiment that validates it.

Table 3: Mapping from RQ claim to validating experiment.

	Claim	Experiment
RQ1	Four families produce checkable proofs; tampered batches rejected	§6.3
RQ2-a	Per-record update ratio grows with n vs. optimised index	§6.4 (n sweep)
RQ2-b	Prover sub-linear in s, linear in m, T	§6.4 (s, m sweep)
RQ3-a	SDC sort-share reduction $\propto K$	§6.5 (vary K)
RQ3-b	HPP collapses $\Theta(md \cdot H)$ to $\Theta(md \cdot L)$ above crossover	§6.5 (vary m, d, N)
RQ3-c	NDC matches additive cost on delete-aware batches	§6.5 (vary f)
RQ3-d	CCS gives $(\varepsilon_N, \delta_N)$ leakage at $\sim 1\%$ extra cost	§6.5 (vary σ)
RQ3-e	Planner layouts differ across profiles with measurable gap	§6.5 (3 profiles)
RQ3-f	Sketch gates outperform zkVM by $\geq 10\times$ on circuit cost	§6.5 (RISC0/SP1)
RQ3-g	Wide-output Poseidon2 cuts Poseidon2 invocations from md to m	§6.5 (vary d)
RQ3-h	LogUp routing matches sort+segment at $\sim 30\%$ fewer column families	§6.5 (vary m)
RQ3-i	Batch dedup cuts hash cost $4\times$–$8\times$ on Zipf-1	§6.5 (vary α)
RQ4-a	Streaming forest's amortised $O(1)$ overtakes the chain at high b	§6.6 (vary b, 1 core)
RQ4-b	Tree and forest scale across cores; chain stays single-threaded	§6.6 (vary b, 16 cores)
RQ4-c	Eager peak chain attains $O(1)$ query latency above the crossover q/b	§6.6 (vary q)

Table 4: Positioning of zkSketch against the closest related systems. Trust: trust model on the prover/server side. Proof: cryptographic output. State: how committed state evolves. Sketches: which sketch families are supported. Backend: cryptographic substrate.

System	Trust	Proof	State	Sketches
zkSketch (this paper)	malicious 1-prover	NIZK, transferable	dynamic, tree-bound	4 families
PoneglyphDB	malicious 1-prover	NIZK, transferable	static committed	—
ZKSQL	malicious 1-prover	NIZK, transferable	static committed	—
zkCLEAR	malicious 1-prover	NIZK, transferable	static / per-DAG	—
S^3	semi-honest 3-server MPC	MPC interactive	streaming	5 families
Whisper	honest-majority 2-server	silently verifiable batch	epoch-batch	heavy hitters

7 Related Work

Table **??** of §1 groups prior work by which of (R-1)–(R-4) each family meets; we use this section to walk those families capability-by-capability and to call out the specific techniques zkSketch borrows or differs from. Table 4 gives a complementary capability-driven snapshot.

zkSketch is the only system in Table 4 that combines (i) malicious single-prover non-interactive ZK with (ii) dynamic committed sketch state bound by a maintained sketch-merge segment tree (§4.3) across (iii) the canonical four sketch families. ZK databases share the trust model and proof type but commit to static relational state; MPC and silently-verifiable systems share the algorithmic content but use a fundamentally different cryptographic regime.

ZK databases for exact data. IntegriDB [43] and vSQL [42] address verifiable SQL queries (vSQL using GKR-style protocols). ZKSQL [19] provides efficient ZK query evaluation over committed relational data. PoneglyphDB [14] gives a non-interactive Halo2-based ZK database with PLONKish gates for SQL operators. VPJoin [2]

shows witness-guided verification of multi-way joins and proposes DP-guided capacities for sensitive intermediate sizes. These systems serve as the optimized exact-index baseline of Appendix B: they are very effective on point queries but do not commit to compact aggregate-friendly summaries, so aggregate queries cost $\Theta(n)$. zkSketch complements rather than replaces them and adapts VPJoin's DP capacity idea to sketch touched-cell cardinalities through CCS.

Domain-specific ZK frameworks. zkCLEAR [37] layers ZK causal inference into backend / operator / module / workflow tiers and decomposes end-to-end workflows into commitment-bounded sub-DAGs with modular proof generation. ZKML [6], zkPyTorch [29], EZKL [44], and zkLLVM [24] build optimizing compilers from ML models or LLVM IR to halo2 circuits. zkSketch borrows the layered-framework pattern from zkCLEAR and the optimizing-compiler pattern from ZKML/zkPyTorch, and contributes deployment-workload awareness for streaming sketch maintenance, the three-dimensional cost surface, and the recursive-SNARK composition design space (§4.3, §**??**).

MPC sketches and streaming ZK. S^3 [40] designs efficient sketch protocols in outsourced MPC and is the closest algorithmic precedent: its batch-update insight (sort, segmented prefix-sum/max, compact, top-s retain, GK fusion rounds) reappears throughout our certificate-checking pipeline. zkSketch differs on four dimensions: *trust model* (S^3 is multi-server semi-honest MPC; zkSketch is single-prover non-interactive ZK against malicious provers), *output shape* (secret shares vs. transferable proofs), *state* (fresh per-batch vs. committed and tree-bound), and *cost model* (MPC rounds and bandwidth vs. PLONKish constraints and proof size). The algorithmic primitives transfer; the cryptographic regime is fundamentally different. Whisper [30] combines streaming, sketching, and silently verifiable proofs with multiple non-colluding servers; streaming ZK proofs provide foundations for low-space verifiers [7]; both target high-throughput streaming with multi-party trust and are not direct comparators for the dynamic single-prover regime.

Verification-over-computation as a technique. Mystique [39], zkCNN [20], and vCNN [18] apply VoC pointwise (Freivalds' algorithm [10]) to one or two operators each. zkSketch elevates the technique to a framework discipline organized by certificate shape (§**??**).

Sketches and AQP. Count-Min, HyperLogLog, SpaceSaving, and GK are classic compact summaries [8, 9, 12, 22]; KLL [15] and the broader mergeable-summaries class [1] are natural extensions; Apache DataSketches packages many in production form [3]. zkSketch differs by adding committed sketch state, a maintained sketch-merge segment tree for verifiable history binding, and verifiable updates over a single untrusted prover.

Recursion, folding, and proof pipelines. Recursive proofs, folding, IVC, and accumulation [5, 16, 17, 32] carry validity across long computations. zkSketch treats them as first-class composition primitives: the three composition topologies of §4.3 place chained / folding, binary-tree, and k-ary aggregation in one design space and characterise the trade-offs against batch size, parallelism, and core count. Reckle Trees [26] apply a single Merkle-tree topology to a static rollup; modern zkVMs and proof-pipeline aggregators [25, 27, 31, 33, 34] adopt k-ary recursion for general-purpose computation. zkSketch is, to our knowledge, the first work to characterise these topologies for the specific workload of streaming sketch maintenance under non-interactive ZK.

8 Discussion and Future Extensions

Comparative crossover and hybrid query plans. The comparative analysis (Appendix B) gives asymptotic costs, but the practical crossover between exact index and committed sketch depends on workload mix and concrete constants H, L, n. A natural extension is a hybrid planner that picks the optimized exact index for point lookups and exact-equality tests and the optimized sketch for aggregates and arbitrary-range frequencies; the framework supports this natively because each strategy is a separate L4 module under the layered architecture (§**??**). The planner of §**??** extends to a per-query-type dispatcher with the same three-dimensional cost surface.

Hash profiles, capacity DP composition, and query privacy. Three privacy-and-deployment concerns interact with the framework's L5 compositional operators. *Hash profiles*: HPP raises the deployment question of who builds and commits the hash table; *beaconed HPP* (deriving the profile from a public randomness beacon) is the cleanest answer when adversarial profile substitution is a concern. *Capacity DP composition*: CCS's DP accountant must compose with any release-policy DP accountant, and the planner of §**??** could be extended to optimize the joint allocation under a global (ε, δ) ceiling. *Public vs. private query keys*: a query for a public key exposes the bucket addresses $b_i = h_i(q)$; private keys preserve more information against curious verifiers but pay more in opening cost. The current decode circuits accept either; the cost-vs-leakage trade-off is a deployment knob.

Deletes for non-linear sketches and group-by formalization. HLL, SS, and GK need policy choices for deletion—soft-delete vs. recompute-on-threshold (§**??**); the planner could pick automatically based on observed deletion rate. zkSketch also treats group-by sketches via a public group-key column g and capacity G; each (g, τ) pair shares SDC-amortized sort/tree-append work with sibling groups. A formal treatment of grouped-decode under merge proofs (does a query spanning several groups preserve the per-group accuracy bound?) is open; lifting the framework interface from Π_τ to $\Pi_\tau \times \text{Group}$ is a natural direction.

Operational concerns and backend extensions. Replay protection: the segment-tree root commitment $R^{(t)}$ enforces *order* across history (any reordering produces a different root under collision resistance and recursive-SNARK knowledge soundness), but not *freshness* (the absence of stale-proof replay). Freshness is a separate concern that requires an external anchor—timestamping, a randomness beacon, or a signed checkpoint—which composes with the tree-root commitment at audit time. *Parameter migration*: a bulk-load proof primitive that re-commits state at widened (d, w) generalizes GK's capacity-threshold rebuild. *Formal verification*: SDC's tag-aware permutation and CCS's coupling permutation are targets for property-based tests [36]. *Backend extensions*: alternative recursive-SNARK substrates (Plonky2 [28], Halo2 [41], SNARK

aggregators) compose with the framework without changes to circuit design; the composition-topology choice (§4) is orthogonal to substrate choice.

9 Conclusion

zkSketch is a verifiable approximate-query-processing system for streaming sketch maintenance over committed datasets, built around committed sketch states with ZK-certified updates and queries. Each new committed batch advances the committed sketch state under a self-contained per-batch subproof; queries are decoded under separate proofs. The per-batch subproofs are composed online via recursive-SNARK composition (§4.3) under a chosen topology (chained / folding, binary tree, or k-ary); the composition root commitment $R^{(t)}$ summarises the verifiable history and the end-of-stream root proof is checkable in $\Theta(1)$ work irrespective of k. Three certificate paradigms share this outer envelope and are chosen per sketch by algebraic structure: sparse-delta with homomorphic-additive vector commitment for Pattern A linear sketches; sparse-delta with batched multi-opening for Pattern A max sketches; mergeable-summary batch certificate for Pattern B sketches. Four compositional techniques (SDC, HPP, NDC, CCS) close the practical gap. Framework-level mechanisms turn the per-circuit baseline into a deployment-grade system. The framework is honest about its scope: ZK certifies exact adherence to the declared sketch algorithm, not exact equality with the true answer; approximation error follows from sketch theory; input completeness follows from authenticated input logs that ZK alone cannot supply; and the privacy property delivered is circuit obliviousness plus DP-bounded capacity leakage. Three structural limits (GK $\log s'$ depth, HLL non-field estimator operations, dense Pattern B whole-state recommit) we leave for future work.

Deployment recommendations. Three deployment patterns warrant explicit recommendations based on the cost surface of §**??**. *Regulatory filings* (audit-grade): use NativeHash + static capacity + per-batch self-contained proofs; verifier scrutiny is high and prover budget is modest, so the per-batch transferable-proof model with no capacity leakage is the right point. *Telemetry archive / blockchain analytics* (throughput-grade): use HPP-LookupHash + adaptive capacity + periodic merge proofs at $E \geq 10^3$ batches per audit; the high-batch-rate workload amortizes HPP preprocessing and merge cost over many queries. *Biomedical analytics or other sensitive-cardinality settings* (leakage-tight): use NativeHash + DP-CCS at $(\varepsilon, \delta) = (1, 10^{-6})$ per audit window + static slot choices; the DP-CCS overhead is small and the formal accounting protects against adversarial cardinality inference. The framework gives the planner the tools to pick within these regimes; the system designer chooses the regime based on the deployment's threat model and workload.

Closing. The framework moves the verifiable-sketch contribution from a per-circuit gate-design exercise to a system-design exercise. New sketch families plug in by filling the seven Π_τ slots and mapping each operator to a row of the certificate catalog (Table **??**)—KLL [15], counting Bloom variants, MinHash, t-digest, and other mergeable summaries [1] are direct extensions covered by the same two-pattern dichotomy. New deployment workflows compose at L6 from existing L5 operators; new backends replace L1 transparently. The four families instantiated here cover the analytical workloads of plaintext AQP libraries [3].

References

[1] Pankaj K. Agarwal, Graham Cormode, Zengfeng Huang, Jeff M. Phillips, Zhewei Wei, and Ke Yi. 2012. Mergeable Summaries. In *Proceedings of the 31st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS)*. 23–34. https://doi.org/10.1145/2213556.2213562

[2] Anonymous Authors. 2026. VPJoin: Verifiable and Privacy-Preserving Multi-Way Join via Zero-Knowledge Proofs. Manuscript supplied by the authors.

[3] Apache Software Foundation. 2026. Apache DataSketches. https://datasketches.apache.org/ Accessed 2026-05-03.

[4] Jon Louis Bentley and James B. Saxe. 1980. Decomposable Searching Problems I: Static-to-Dynamic Transformation. *Journal of Algorithms* 1, 4 (1980), 301–358.

[5] Benedikt Bünz and Binyi Chen. 2023. ProtoStar: Generic Efficient Accumulation/Folding for Special-Sound Protocols. In *Advances in Cryptology – ASIACRYPT 2023*. 77–110. https://doi.org/10.1007/978-981-99-8724-5_3

[6] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. 2024. ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*. 560–574. https://doi.org/10.1145/3627703.3650088

[7] Graham Cormode, Marcel Dall'Agnol, Tom Gur, and Chris Hickey. 2024. Streaming Zero-Knowledge Proofs. In *39th Computational Complexity Conference (LIPIcs)*, Vol. 300. 2:1–2:66. https://doi.org/10.4230/LIPIcs.CCC.2024.2

[8] Graham Cormode and S. Muthukrishnan. 2005. An Improved Data Stream Summary: The Count-Min Sketch and Its Applications. *Journal of Algorithms* 55, 1 (2005), 58–75. https://doi.org/10.1016/j.jalgor.2003.12.001

[9] Philippe Flajolet, Eric Fusy, Olivier Gandouet, and Frederic Meunier. 2007. HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm. In *Analysis of Algorithms 2007*. 127–146. https://algo.inria.fr/flajolet/Publications/FlFuGaMe07.pdf

[10] Rūsiņš Freivalds. 1977. Probabilistic Machines Can Use Less Running Time. *Information Processing 77, Proceedings of IFIP Congress 77* (1977), 839–842.

[11] Ariel Gabizon and Zachary J. Williamson. 2020. Plookup: A Simplified Polynomial Protocol for Lookup Tables. IACR Cryptology ePrint Archive, Report 2020/315. https://eprint.iacr.org/2020/315

[12] Michael Greenwald and Sanjeev Khanna. 2001. Space-Efficient Online Computation of Quantile Summaries. In *Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data*. 58–66. https://doi.org/10.1145/375663.375670

[13] Jens Groth. 2016. On the size of pairing-based non-interactive arguments. In *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 305–326.

[14] Binbin Gu, Juncheng Fang, and Faisal Nawab. 2025. PoneglyphDB: Efficient Non-Interactive Zero-Knowledge Proofs for Arbitrary SQL-Query Verification. *Proceedings of the ACM on Management of Data* 3, 1, Article 63 (2025). https://doi.org/10.1145/3709713

[15] Zohar Karnin, Kevin Lang, and Edo Liberty. 2016. Optimal Quantile Approximation in Streams. In *Proceedings of the 57th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*. 71–78. https://doi.org/10.1109/FOCS.2016.17

[16] Abhiram Kothapalli and Srinath Setty. 2022. SuperNova: Proving Universal Machine Executions without Universal Circuits. IACR Cryptology ePrint Archive, Report 2022/1758. https://eprint.iacr.org/2022/1758

[17] Abhiram Kothapalli and Srinath Setty. 2023. HyperNova: Recursive Arguments for Customizable Constraint Systems. IACR Cryptology ePrint Archive, Report 2023/573. https://eprint.iacr.org/2023/573

[18] Seunghwa Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. 2024. vCNN: Verifiable Convolutional Neural Network Based on zk-SNARKs. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2024), 4254–4270. https://doi.org/10.1109/TDSC.2023.3348760

[19] Xiling Li, Chenkai Weng, Yongxin Xu, Xiao Wang, and Jennie Rogers. 2023. ZKSQL: Verifiable and Efficient Query Evaluation with Zero-Knowledge Proofs. *Proceedings of the VLDB Endowment* 16, 8 (2023), 1804–1816. https://doi.org/10.14778/3594512.3594513

[20] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2968–2985. https://doi.org/10.1145/3460120.3485379

[21] Claire Mathieu, Rajmohan Rajaraman, Neal E. Young, and Arman Yousefi. 2024. Competitive Data-Structure Dynamization. *ACM Transactions on Algorithms* 20, 4 (2024).

[22] Ahmed Metwally, Divyakant Agrawal, and Amr El Abbadi. 2006. An Integrated Efficient Solution for Computing Frequent and Top-k Elements in Data Streams. *ACM Transactions on Database Systems* 31, 3 (2006), 1095–1133. https://doi.org/10.1145/1166074.1166084

[23] Mina Foundation. 2024. Scan State: Parallel Recursive Proof Composition for the Mina Protocol. https://docs.minaprotocol.com/mina-protocol/scan-state.

[24] =nil; Foundation. 2024. zkLLVM Circuit Compiler. https://github.com/NilFoundation/zkllvm Accessed 2026-05-03.
[25] OpenVM. 2026. Generating Proofs. https://docs.openvm.dev/book/writing-apps/generating-proofs/ Accessed 2026-05-03.
[26] Charalampos Papamanthou, Shravan Srinivasan, Nicolas Gailly, Ismael Hishon-Rezaizadeh, Andrus Salumets, and Stjepan Golemac. 2024. Reckle Trees: Updatable Merkle Batch Proofs with Applications. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM. https://doi.org/10.1145/3658634.3670354
[27] Polygon Labs. 2026. Polygon zkEVM Proving Architecture. https://docs.polygon.technology/tools/zkevm/architecture/zkprover/stark-recursion/proving-architecture/ Accessed 2026-05-03.
[28] Polygon Zero. 2022. Plonky2: Fast Recursive Arguments with PLONK and FRI. https://github.com/0xPolygonZero/plonky2/blob/main/plonky2/plonky2.pdf Whitepaper, accessed 2026-05-03.
[29] Polyhedra Network. 2025. zkPyTorch: A Hierarchical Optimized Compiler for ZK Machine Learning. IACR Cryptology ePrint Archive, Report 2025/535. https://eprint.iacr.org/2025/535
[30] Mayank Rathee, Yuwen Zhang, Henry Corrigan-Gibbs, and Raluca Ada Popa. 2024. Private Analytics via Streaming, Sketching, and Silently Verifiable Proofs. IACR Cryptology ePrint Archive, Report 2024/666. https://eprint.iacr.org/2024/666
[31] RISC Zero. 2026. Recursive Proving Documentation. https://dev.risczero.com/api/recursion Accessed 2026-05-03.
[32] Srinath Setty. 2022. Nova: Recursive Zero-Knowledge Arguments from Folding Schemes. In *Advances in Cryptology – CRYPTO 2022*. 359–388. https://doi.org/10.1007/978-3-031-15985-5_13
[33] Starknet. 2026. SHARP: Starknet Proof Aggregator Documentation. https://docs.starknet.io/learn/protocol/sharp Accessed 2026-05-03.
[34] Succinct. 2026. SP1 Proof Aggregation Documentation. https://docs.succinct.xyz/docs/sp1/writing-programs/proof-aggregation Accessed 2026-05-03.
[35] Peter Todd. 2016. Merkle Mountain Ranges. OpenTimestamps technical specification. https://github.com/opentimestamps/opentimestamps-server/blob/master/doc/merkle-mountain-range.md
[36] Trail of Bits. 2025. A Deep Dive into Axiom's Halo2 Circuits. https://blog.trailofbits.com/2025/05/30/a-deep-dive-into-axioms-halo2-circuits/ Accessed 2026-05-03.
[37] Zhaoyu Wang, Pingchuan Ma, Zhantong Xue, Yanbo Dai, Zhenlan Ji, and Shuai Wang. 2025. Privacy-Preserving and Verifiable Causal Prescriptive Analytics. *Proceedings of the ACM on Management of Data* 3, 6, Article 350 (2025). https://doi.org/10.1145/3769815
[38] Zhaoyu Wang, Pingchuan Ma, Zhantong Xue, Yuguang Zhou, Qixin Zhang, Xiaoqin Zhang, and Shuai Wang. 2026. ZK-Value: A Practical Zero-Knowledge System for Verifiable Data Valuation. arXiv preprint arXiv:2605.03581. https://arxiv.org/abs/2605.03581
[39] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. 2021. Mystique: Efficient Conversions for Zero-Knowledge Proofs with Applications to Machine Learning. In *30th USENIX Security Symposium*. 501–518.
[40] Jianzhe Yu, Feng Han, Qi Dong, Qiyao Luo, Weiran Liu, Lin Qu, and Ke Yi. 2026. Sketch-Based Secure Query Processing for Streaming Data. *Proceedings of the ACM on Management of Data* 4, 3, Article 233 (2026). https://doi.org/10.1145/3802110
[41] Zcash Foundation. 2026. The halo2 Book. https://zcash.github.io/halo2/ Accessed 2026-05-03.
[42] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. 2017. vSQL: Verifying Arbitrary SQL Queries over Dynamic Outsourced Databases. In *2017 IEEE Symposium on Security and Privacy*. 863–880. https://doi.org/10.1109/SP.2017.48
[43] Yupeng Zhang, Jonathan Katz, and Charalampos Papamanthou. 2015. IntegriDB: Verifiable SQL for Outsourced Databases. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 1480–1491. https://doi.org/10.1145/2810103.2813711
[44] zkonduit. 2026. EZKL: A Library and Command-Line Tool for Verifiable Inference of ML Models. https://docs.ezkl.xyz/ Accessed 2026-05-03.

A Framework Guarantees

The framework's guarantees follow by composing local transition soundness with recursive history soundness.

Local transition soundness. For every update j, the per-update proof π_j is knowledge-sound for $\mathcal{R}^{\tau}_{\mathsf{upd}}$. Therefore, an accepting proof binds $\mathsf{Com}(S_j)$ to $\mathsf{Com}(S_{j-1})$ and C_{b_j} under the declared update algorithm. A prover who fabricates the new state, omits a record from the committed update, or applies the wrong sketch algorithm cannot produce an accepting proof except with negligible probability.

History soundness. For every interval $[i, j]$, the recursive proof $\Pi_{[i,j]}$ verifies the child interval proofs and enforces equality of adjacent boundary state commitments. By induction on the interval tree, the root proof $\Pi_{[1,t]}$ certifies that $\mathsf{Com}(S_t)$ is reachable from $\mathsf{Com}(S_0)$ by applying the committed updates $b_1, \ldots, b_t$ in order.

Query-prefix soundness. For a query at logical time t, the prover first closes all uploads before t into committed updates and proves the resulting history root. The decode proof then certifies the answer against the corresponding state commitment. Hence an accepting query proof is tied to exactly the uploaded prefix visible to that query.

Tamper resistance. Dropping an update, reordering two updates, replacing an update commitment, or substituting an intermediate state changes either a leaf public input, an interval boundary, or the ordered digest $H_{[1,t]}$. Under commitment binding and hash collision resistance, such a change cannot produce the same root commitment $R^{(t)}$ with an accepting root proof, except with negligible probability.

Succinct verification. The verifier checks only the root proof $\Pi^{(t)}$ against the root commitment $R^{(t)}$, the current state commitment $\mathsf{Com}(S_t)$, and the public metadata. Verification cost is independent of the number of updates in the verification window and independent of the number of records inside those updates.

Privacy. Zero knowledge hides the update witnesses, including raw records, sketch states, delta tables, sorted certificates, and commitment openings. Circuit obliviousness ensures that proof shape depends only on public capacities and parameters. Thus, the verifier learns the declared metadata, the committed history root, and the validity of the certified statements, but not the private records or private intermediate states.

B Design Space and Cost Derivations

This appendix has three roles: (i) it walks the plaintext Count-Min update inside ZK and quantifies the per-cost-axis pains on the running example (the *warm-up*, formerly §4.1 in earlier drafts); (ii) it names and quantifies the `naive-zkSketch` strawman as the cost to beat (formerly §4.2); and (iii) it places committed sketches relative to exact ZK indexes through a compact cost comparison.

Warm-up: plaintext Count-Min inside ZK.. Count-Min's plaintext update is a one-liner on a record (k, v): compute d keyed hashes $h_i(k)$ and increment d cells $S[i, h_i(k)]$ += v. Inside a ZK circuit this becomes a sequence of d in-circuit Poseidon evaluations ($\sim$ 300 constraints each), d private-address Merkle paths ($\sim 2 \log w \times 300 \approx 4{,}800$ constraints per record at $\log w = 8$), within-batch collision sequencing, and a state recommit. On a production deployment ($m = 1024, d = 7, w = 2^{16}, H \approx 300$) the direct encoding costs $\sim 3.4 \times 10^7$ constraints per batch — an order of magnitude above the optimised pipeline of §4 + §5. Three challenges crystallise the gap: **C1** private random writes at hash-determined addresses; **C2** within-batch collisions resolved in data-dependent order; **C3** capacity-dependent leakage (touched-cell count is a function of

Table 5: Asymptotic PLONKish cost per query / per single-record update for naive and optimized baselines, ignoring lower-order constants. Per-row derivations and per-family naive baselines for sketches other than CM appear in Appendix C. Boldface entries highlight where each baseline wins.

	Naive		Optimized	
Operation	Index	Sketch	Index	Sketch
Point query	$\Theta(H \log n)$	$\Theta(dH \log w)$	$\Theta(L)$	$\Theta(dL)$
Range freq.	$\Theta(RH \log n)$	$\Theta(dH \log w)$	$\Theta(RL)$	$\Theta(dL)$
Top-k	$\Theta(nH \log n)$	$\Theta(sH)$	$\Theta(nL)$	$\mathbf{\Theta(sL)}$
Distinct	$\Theta(nH \log n)$	$\Theta(sH)$	$\Theta(nL)$	$\mathbf{\Theta(sL)}$
Quantile	$\Theta(nH \log n)$	$\Theta(s'H)$	$\Theta(nL)$	$\mathbf{\Theta(s'L)}$
Per-rec. update	$\Theta(H \log n)$	$\Theta(dH \log w)$	$\Theta(H \log n)$	$\mathbf{\Theta(d + L)}$

key skew). The framework's delta certificate, HPP, and capacity-bounded padding address C1, C2, and C3 respectively.

Strawman: `naive-zkSketch` *and the cost to beat.* We call the direct encoding above `naive-zkSketch`. It pays for three things the actual sketch update does not need to certify: per-record per-row Merkle paths ($\Theta(md)$), in-circuit Poseidon ($\Theta(mdH)$), and full-state commitment recompute ($\Theta(s)$). The delta-certificate construction of §5 replaces all three: the prover groups writes outside the circuit and certifies the grouping by permutation/sortedness/segment aggregation ($\Theta(md)$ rows verified, not paths opened); HPP (§**??**) replaces in-circuit Poseidon with committed lookups ($\Theta(mdH) \to \Theta(mdL)$, $L \ll H$); the homomorphic-additive vector commitment reduces the recommit to the touched cells ($\Theta(T)$ instead of $\Theta(s)$). For the running parameters with $T = 64$ and $L \approx 30$, the post-optimisation per-batch cost is around 4×10^5 constraints — an $\sim 80\times$ saving over `naive-zkSketch`. Two further circuit-level optimisations from §5.4 stack on top when their preconditions hold: wide-output Poseidon2 (§5.4) cuts the in-circuit hash invocations under NativeHash from md to m, a further $d\times$ reduction on the hash share when LookupHash is not chosen; and batch deduplication (§5.4) cuts those m invocations to u unique-key permutations under skewed workloads (4–8× on Zipf-$\alpha = 1$ streams). The multiset-equality routing alternative (§5.4) changes the column-family count rather than the asymptotic, so it does not move the headline 80× but shifts constants the planner of §**??** can exploit. We do not implement `naive-zkSketch` as an evaluation column in §6; it is a rhetorical cost benchmark that makes the framework's structural savings explicit.

Comparative cost analysis. The rest of this appendix places committed sketches relative to exact ZK indexes through a compact cost comparison in PLONKish constraint counts and derives every entry of Table 5 from a per-row PLONKish accounting.

Setting. A committed dataset D of n records evolves through batches $B_1, B_2, \ldots$ of m records each. We compare an *exact index* (sorted-index commitment with lookup arguments, in the methodology of PoneglyphDB and ZKSQL [14, 19]) against a *committed sketch* maintained by the delta-certified design of §4. Each strategy admits a naive PLONKish encoding (replay the plaintext computation) and an optimized one. H denotes per-Poseidon-evaluation cost; L denotes per-lookup-argument cost.

Three observations carry the design space. *(1) Point queries are comparable across optimized baselines*: optimized index pays one lookup; optimized sketch pays d openings. *(2) Aggregate queries asymptotically favor the optimized sketch*: top-k, distinct, quantile, arbitrary-range frequency all run in $\Theta(s \cdot L)$ or $\Theta(s' \cdot L)$ vs. $\Theta(n \cdot L)$ for the index. *(3) Sketches do not universally dominate*: exact-equality tests and per-record retrieval are the index's authority.

Naive PLONKish baselines. Naive exact index. Per-record update is a $\log n$-deep Merkle path with one Poseidon per level: $\Theta(H \log n)$. Per point query: $\Theta(H \log n)$. Per range-frequency of R keys: $\Theta(RH \log n)$. Per top-k/distinct/quantile: $\Theta(nH \log n)$. *Naive sketch (CM).* Per record: d in-circuit hashes ($\Theta(dH)$) + d Merkle openings ($\Theta(dH \log w)$) + d rebindings + collision sequencing. Total: $\Theta(dH \log w)$. Aggregate decode against the committed sketch state: $\Theta(sH)$, independent of n.

Optimized PLONKish baselines. Optimized exact index. Per point query: one lookup ($\Theta(L)$); per range: $\Theta(RL)$; per top-k/distinct/quantile: $\Theta(nL)$. Per-record update under authenticated balanced-tree maintenance: $\Theta(H \log n)$. *Optimized sketch.* Delta-certified maintenance gives per-record update $\Theta(d + L)$ amortized; aggregate queries cost $\Theta(sL)$ or $\Theta(s'L)$.

Streaming regime. The per-record update ratio between optimized exact index and optimized sketch is $\Theta(H \log n/(d+L))$, which grows with n for any fixed d and reasonable H, L. At $H \approx 200$, $L \approx 5$, $d = 7$, $n = 2^{30}$, the ratio is ≈ 500. The streaming sketch-maintenance regime separates the two strategies on every per-update workload regardless of query mix.

C Naive Baselines by Sketch Family

The naive PLONKish baseline of Appendix B uses CM as the canonical example. The other families pay differently. *CM*: $\Theta(dH \log(dw))$ per record. *HLL*: naive update is one hash + one max-rebind ($\Theta(H \log s)$), but decode scans every register ($\Theta(sH)$). *SS*: per record $\Theta(sH)$ for membership probe + argmin scan + branch + evict/insert, with data-dependent control flow. *GK*: per record $\Theta(s'H)$ for sorted-position search + insertion shifts + periodic compression. The optimized delta-certified design of §4 collapses these heterogeneous naive baselines into one uniform certificate-verification pattern.

D Framework Interface and Slot Catalog

Interface tuple. A sketch is integrated by specifying $\Pi_\tau = (\Sigma_S, \Sigma_\Delta, \oplus, \mathsf{Patch}, \mathsf{Deco}$ together with PLONKish realizations. Σ_S: committed layout of S_t. Σ_Δ: per-record certificate row shape. $\oplus$: bounded-depth aggregation/merge operator over delta rows targeting the same address (Pattern A) or shard (Pattern B). Patch: sparse cell rebinding (Pattern A) or whole-state replacement (Pattern B). Decode_q: query rule. Merge: combine compatible committed states. Compat: public predicates two states must satisfy to be merge-compatible.

Compiler. The deployment-aware sketch compiler is presented as Algorithm **??** in §**??**; per-sketch planner outputs across the three deployment profiles are tabulated in Table **??** of the same subsection.

Table 6: L3 slot catalog. The planner selects a realization per slot at compile time. H = per-Poseidon cost; L = per-lookup cost; T = touched-cell count; $|S|$ = state size; N = HPP table size.

Slot	Realizations	Prover	Verifier
HashProfile	NativeHash (Poseidon / Poseidon2 / Anemoi / Monolith)	$\Theta(H)$	$O(1)$
	LookupHash (Plookup / LogUp / CQ)	$O(\log N)$	$\Theta(N)$ once
	HashCert (external Merkle)	$\Theta(\log N)$	$\Theta(\log N)$
SegSum / SegMax	Local recurrence	$\Theta(T)$	$O(1)$
	Prefix-difference	$\Theta(T)$	$O(1)$
Patch	SparseAddPatch + MSMConsistency (A linear)	$\Theta(T)$	$O(1)$
	SparseMaxPatch + IPA/KZG (A max)	$\Theta(T + \log \lvert S\rvert)$	$\Theta(\log \lvert S\rvert)$
	ReplaceState whole-state recommit (B)	$\Theta(\lvert S\rvert)$	$O(1)$
	Lookup-argument-backed table	$\Theta(T)$ lookups	$\Theta(\lvert S\rvert)$ once
TopS / FuseTuple	Threshold-residual certificate (TopS)	$\Theta(s)$	$O(1)$
	$\log s'$ pairwise rounds (FuseTuple)	$\Theta(s' \log s')$	$O(1)$
Capacity	Static / Adaptive / DP-via-CCS	varies	varies

E Detailed Gate Specifications

This appendix gives witness/output/verification-target/realization specifications for the gates introduced or adapted by zkSketch, in the format used by PoneglyphDB and VPJoin [2, 14]. Established primitives (RangeCheck, IsZero, IsEq, Cmp, Mux, MinMax, ArgMin, ArgMax, PrefixSum, PrefixMax, Permute, LexSort, ActiveFirst, Segment, SegSum, SegMax, StableCompact, Disjoint) follow their canonical realizations and are not detailed here.

E.1 Canonical Table Encodings

All sketch gates use a small number of table encodings. We state them explicitly because most soundness bugs in PLONKish database circuits arise from missing active flags, dummy rows, or permutation columns. *Active rows and dummy rows.* Every witness table has an active bit a_j. The circuit enforces $a_j(a_j - 1) = 0$. For a tuple $t_j = (a_j, x_{j,1}, \ldots, x_{j,r})$, inactive rows must equal a fixed dummy: $(1 - a_j)(x_{j,\ell} - \perp_\ell) = 0$ for every field ℓ. *Compressed tuple encoding.* For permutation checks, a row is compressed as $E_j = tag_j + \alpha_0 a_j + \alpha_1 x_{j,1} + \cdots + \alpha_r x_{j,r}$ where the α_i are verifier challenges; tags distinguish tables sharing columns. The circuit uses a permutation or grand-product argument to prove multiset equality of encoded rows. *Sorted tuple keys.* Lexicographic order on a multi-column key is enforced by equality-prefix bits $p_{j,\ell} = p_{j,\ell-1} \cdot [k_{j,\ell} = k_{j+1,\ell}]$ and less-than bits $q_{j,\ell} = [k_{j,\ell} \le k_{j+1,\ell}]$, with the order condition $\sum_\ell p_{j,\ell-1}(1 - [k_{j,\ell} = k_{j+1,\ell}])q_{j,\ell} + p_{j,r} = 1$. In practice the key may be packed into a bounded integer when its packed range fits the field, replacing the multi-column comparison with one range check on the difference. Realized in Halo2 as advice columns plus permutation-argument checks; the equality-prefix and less-than bits are degree-2 custom gates [41].

E.2 Commitment Scheme Choice

A naive Merkle-tree commitment costs $\Theta(\log |S| \cdot H)$ per cell access, dominating the update circuit when T is in the thousands. The framework recommends:

Table 7: Recommended commitment schemes per pattern.

Pattern	Sketches	Scheme	Per-batch cost
A linear	CM	Homomorphic-additive vector commitment	$O(T) + O(1)$
A max	HLL	Vector commitment + batched IPA/KZG	$O(T + \log \lvert S\rvert)$
B	SS, GK	Vector commitment over bounded state	$O(\lvert S\rvert)$ MSM
Any (medium)	state $\le 2^{20}$	Lookup-argument-backed table	$O(T)$ lookups

E.3 TopS

TopS$(G, s, \prec, N, R)$. **Witness.** Grouped table G, kept table N of capacity s, residual table R, public total order $\prec$. **Verification target.** N contains the s best active rows of G under $\prec$. **Realization.** (i) Conservation $G \equiv N \cup R$ via Permute; (ii) N sorted by $\prec$; (iii) every active residual $r \in R$ no better than the last active row of N (or no better than a dummy threshold), checked by Cmp against a witness threshold τ. Realized in Halo2 as one permutation argument over compressed-tuple advice columns plus per-residual range-check on the Cmp difference; circuit cost is linear in $|G| + |N| + |R|$, which is asymptotically tighter than any oblivious top-s sorting network [41].

E.4 HashProfile

HashProfile(P, i, k). Returns $b \in [0, w)$ with $b = h_i(k)$ under public profile digest P. Three modes: *NativeHash* (in-circuit ZK-friendly hash), *LookupHash* (membership of (P, i, k, b) in a committed profile table [11, 41]), *HashCert* (external Merkle opening or universal-hash certificate). The HLL LeadingRun variant is detailed in Appendix F.

Three-mode cost profile (HPP).. HPP commits the hash-evaluation table $T_H = \{(P, i, k, h_i(k))\}$ of size N as a public commitment C_H, then verifies per-record hashes by lookup. The three HashProfile modes have different cost profiles. *NativeHash* pays $\Theta(H)$ per record on the prover and $O(1)$ on the verifier; the table C_H is unused. *LookupHash* pays $O(\log N)$ amortized per record on the prover via a Plookup or LogUp lookup against C_H; the verifier pays $\Theta(N)$ once per profile-table commit. *HashCert* offloads hashing to an external certificate (Merkle opening or universal-hash witness), raising verifier cost but minimizing prover cost on long-lived deployments. The deployment chooses: HashCert is most useful when the same profile is reused across many independently maintained sketches; LookupHash is most useful when batch sizes are large

and the profile table fits within Halo2 lookup capacity; NativeHash is the fall-back for ad-hoc deployments. Realized in Halo2 as a Plookup/LogUp argument over advice columns [11, 41]; the per-batch hash share collapses from $\Theta(md \cdot H)$ to $\Theta(m \cdot L)$ under LookupHash, with the $\Theta(N \cdot H)$ one-time preprocessing cost amortized across all subsequent batches under the same profile.

E.5 MSMConsistency

Setting. Fix commitment key $\{G_\kappa\}_{\kappa\in[N]}$ for a vector commitment of capacity $N = |S|$. **MSMConsistency**(A, C_Δ)**.** Witness: aggregate $A = \{(\kappa_j, \Delta_j)\}$ of capacity $T_{\max}$ with active flags. Verification: $C_\Delta = \sum_j a_j \Delta_j G_{\kappa_j}$. Realization in three phases: (i) bucket-key extraction with RangeCheck $\kappa_j < N$ and Mux for $\hat{\Delta}_j = a_j\Delta_j$; (ii) per-row scalar multiplication $P_j = \hat{\Delta}_j G_{\kappa_j}$ via lookup-table key access plus windowed scalar multiplication ($\sim$ 100–200 constraints per scalar under Pasta-cycle Halo2 IPA); (iii) running-sum $Q_j = Q_{j-1} + P_j$ with $Q_{T_{\max}} = C_\Delta$ as a curve-equation check. Cost: $\Theta(T_{\max})$ scalar mults + EC adds, independent of $|S|$.

E.6 State, Commitment, and Tree-Append Gates

OpenCell$(C, \kappa, x, \pi_{open})$. Certifies $C[\kappa] = x$. Under homomorphic-additive vector commitment, implicit in SparseAddPatch's group-relation check; under batched IPA/KZG, amortized into one multi-opening of cost $O(T + \log|S|)$; under Merkle (naive baseline only), depth-log $|S|$ path; under lookup table, single lookup.

SparseAddPatch(C, C', A). Verifies $C'[\kappa] = C[\kappa] + \Delta$ for $(\kappa, \Delta) \in A$. Under the recommended homomorphic-additive vector commitment, the gate verifies $C' = C + \mathsf{Com}(\Delta)$ together with MSMConsistency tying $\mathsf{Com}(\Delta)$ to A; *no per-cell Merkle path required.*

SparseMaxPatch(C, C', A). Same as SparseAddPatch but $x' = \max(x, \Delta)$. The homomorphic shortcut does not apply; recommended realization is a batched IPA/KZG multi-opening plus in-circuit max gates.

ReplaceState(C, C', T, T'). Whole-state replacement under bounded-state vector commitment of Pattern B; recommit cost is one MSM of size $|T'| \le s$ or s'.

MergeTree-Append$(R, R', t, \mathsf{Com}(S_t), \pi_t, M)$. The composition-extension gate for the binary-tree topology (§4.3): enforces that R' is the binary-tree root obtained by appending a leaf at position t carrying $(\mathsf{Com}(S_t), \pi_t)$ to the tree with root R. For merge proofs across shards, the tree-append takes the children's roots as siblings instead of a single appended leaf. Prevents replay, reordering, skipped batches, double-counting, hash-profile substitution. The analogous gates for the other topologies are *Chained-Fold* (§4.3: extends the accumulator A_{t-1} by folding in the new commitments $\mathsf{Com}(S_t), \pi_t$, producing A_t and the implicit root $R^{(t)} = \mathsf{hash}(A_t)$) and *k-ary-Merge* (§4.3: takes k sibling commitments and produces their composed root). Each variant preserves the same soundness profile: replay / reorder / drop attacks change the root commitment except with negligible probability.

Compat. Public-metadata predicates: sketch type, dimensions, key domain, hash/sign profile, counter representation, group schema, deletion policy, release policy, overflow bounds. Merge gates additionally check disjoint partition coverage.

F HyperLogLog (Pattern A specialization)

HLL instantiates Π_τ as Pattern A with a different cell type and aggregator. Σ_S is an s-register array $R[g, b]$ of small integers per group; Σ_Δ rows carry (g, b, ρ) where ρ is the leading-run statistic of a second hash $h_2(k)$; $\oplus$ is max (SegSum becomes SegMax); Patch is SparseMaxPatch; Decode_q is a register-sum or harmonic-mean estimator. The aggregation operator max is associative-commutative but *not* additive, with two consequences for the framework. *Per-record gate.* HLL-Generate evaluates $b = h_1(k)$ and $z = h_2(k)$ via HashProfile and certifies $\rho = \mathsf{LeadingRun}(z)$ via a prefix-and recurrence on the bit decomposition of z. *Patch operator.* The non-additive max rules out the homomorphic shortcut: SparseMaxPatch verifies the patch by a single batched IPA/KZG multi-opening over the T touched registers plus in-circuit max gates. The batched opening costs $O(T + \log|S|)$, the dominant Pattern A max term in Table **??**. Decode supports a raw-statistic mode (cheap) and an in-circuit-estimator mode (audit-grade, expensive due to non-field 2^{-r} lookup and reciprocal); the (F2) limit (§**??**) explains why the in-circuit-estimator cost cannot be eliminated.

LeadingRun(x, ℓ). Witness: bit decomposition $x_1, \ldots, x_L$, prefix bits p_j, output ℓ. Constraints: $p_1 = x_1$, $p_j = p_{j-1} \cdot x_j$, $\ell = \sum_j p_j$. Each bit range-checked.

HLL-Generate. Per record k_j: $b_j = h_1(k_j)$, $z_j = h_2(k_j)$, $\rho_j = \mathsf{LeadingRun}(z_j)$.

HLL-Update. Sort D by (g, b), verify Permute and LexSort, aggregate by SegMax, apply SparseMaxPatch.

HLL-Query. Two release modes. *Raw-statistic*: open all registers in the queried group, prove $Z = \sum_b R[b]$ or $Z = \sum_b 2^{-R[b]}$ via fixed-point lookup; client applies public post-processing. *In-circuit-estimator*: in-circuit fixed-point estimator with reciprocal. The first is the default; the second is audit-grade.

Optimisation 1 — wide-output Poseidon2 (applied). HLL-Generate needs two hash outputs per record: $b_j = h_1(k_j)$ for the register index and $z_j = h_2(k_j)$ for the leading-run statistic. Under NativeHash mode, the wide-output trick of §5.4 produces both from a single Poseidon2 permutation: set the input state to $(k_j, 0, \ldots, 0)$, apply one permutation, and read output positions 0 and 1 as (b_j, z_j) with the appropriate range-check decompositions. This halves the per-record Poseidon2 invocation count on HLL. The construction is the same wide-output sub-mode of HashProfile used for CM; no new gate primitives are introduced.

Optimisation 3 — batch deduplication (applied, workload-conditional). The dedup argument of §5.4 carries over verbatim: the prover deduplicates the batch off-circuit into unique keys U with multiplicities μ, certifies the dedup against C_{B_t} via a multiplicity-weighted LogUp identity, and runs HLL-Generate only on U (size $u \le m$, $d = 2$ outputs each under wide-output Poseidon2). Because HLL aggregates by max, duplicate records produce the same (b, ρ) pair and contribute identically to the SegMax; the multiplicity column is therefore *not* used on the cell side (max ignores it), but the unique-key hashing saves m/u Poseidon2 invocations. Pays off on skewed streams.

Optimisation 2 — LogUp routing (does not apply). The multiset-equality routing of §5.4 replaces sort+segment with a LogUp identity balanced by per-cell sums. HLL's SegMax is not a multiset-equality identity — the cell-side aggregate is a max, not a multiplicity-weighted sum — so LogUp routing does not specialise cleanly. The baseline sort+SegMax of HLL-Update stays the default.

G SpaceSaving (Pattern B specialization)

SpaceSaving [1, 22] instantiates Π_τ as Pattern B with a fixed-capacity top-counter state. Σ_S is a keyed top-counter table of capacity s with rows $(a, key, count, err)$ committed under a vector commitment; Σ_Δ rows carry one record's key and count; $\oplus$ is top-s promote-or-evict; Patch is whole-state replacement under a top-s retention certificate; Decode_q is an exact-match lookup with a one-hot match vector; Merge runs the same top-s retention on the $2s$-row union of two children's tables. The structure parallels GK (§5.5): the prover constructs the post-batch state outside the circuit, and the circuit certifies the result satisfies the mergeable-summary invariant.

SS-Union. Concatenate the active rows of the old state S and the buffer rows of B into a unified table U of capacity $s + m$ with role tag $\in \{\text{old}, \text{new}\}$ via Permute on the compressed encoding.

SS-Group. $\text{LexSort}(U, key)$ with inactive rows last; segment-aggregate the per-key counts via SegSum; StableCompact the grouped rows so each distinct key appears in exactly one active row with the summed count.

SS-TopS.. The *threshold-residual* certificate: the prover supplies the kept top-s table S', the residual R, and a threshold row τ equal to the worst active row of S' under the public total order $\prec$ (decreasing *count*, tie-break ascending *key*). The circuit verifies $G \equiv S' \uplus R$ via Permute on the compressed encoding, certifies S' is LexSorted by $\prec$, extracts τ as the last active row of S' via a position-end selector $\sum_r p_r = 1$, enforces $r \prec \tau$ for every active $r \in R$ via Cmp gated by the active flag, and rebinds $\text{Com}(S_t)$ via ReplaceState. The threshold-residual shape asserts a global "best-s" property by comparing each residual against one threshold instead of sorting all $s + m$ candidate rows in-circuit; without it a malicious prover could omit a high-counter row from S' undetected.

SS-PointQuery. Open all s rows of S', compute a one-hot match vector $\{e_r = [S'[r].key = q]\}$ with $e_r \in \{0, 1\}$, return $ans = \sum_r e_r \cdot S'[r].count$. Distinct-key enforcement in SS-TopS guarantees at most one match. Top-c queries return the first c active rows of S', which is already sorted by $\prec$; cost $\Theta(c)$.

Gate SS-Merge. Form the $2s$-row union $U = S_l \uplus S_r$ via Permute (with role tag $\in$ {left, right} for traceability); resolve same-key collisions by LexSort on *key* and SegSum on *count* (and on *err* via the declared mergeable-summary rule); retain the top-s rows via the SS-TopS threshold-residual certificate; recommit $\text{Com}(S_v)$ via ReplaceState. Cost per internal node: $O(s)$ constraints, reusing the existing TopS primitive.

Optimisation 2 — LogUp routing (applied as an alternative to SS-Group). The multiset-equality routing of §5.4 specialises directly to the SS-Group step: per-key sums are the canonical LogUp identity. The prover supplies the grouped table G directly (sized $s + m$ with deactivated duplicates); the LogUp identity certifies $\sum_{u \in U : u.key = k} u.count = G[k].count$ for every active key $k \in G$. This replaces one Permute, one LexSort, the boundary and end bit witness columns, and the SegSum recurrence with a single LogUp bus, reusing the Plonky3 primitive shared with CM-Update. The downstream SS-TopS gate is unaffected. The planner picks between the baseline sort+segment of SS-Group and LogUp routing on per-deployment constants.

Optimisation 3 — batch deduplication (applied). The dedup argument of §5.4 carries over: on skewed batches the prover deduplicates buffer rows off-circuit into unique keys U with multiplicities μ, certifies the dedup against C_{B_t} via a multiplicity-weighted LogUp identity, and feeds U (size $u \le m$) into SS-Union in place of the full B. SS does not hash records in-circuit, so the saving is in the smaller SS-Union table (rows $s + u$ instead of $s + m$) and the proportionally smaller SS-Group bus rather than in Poseidon2 invocations. Pays off on top-k workloads with skewed key distributions.

Optimisation 1 — wide-output Poseidon2 (does not apply). The wide-output trick of §5.4 compresses multiple in-circuit hash invocations per record into one Poseidon2 permutation. SS uses keys directly without per-record in-circuit hashing, so there are no Poseidon2 invocations to compress.

Cost. The naive online SpaceSaving update would cost $\Theta(sH)$ per record (membership probe, argmin scan, evict, insert), totaling $\Theta(msH)$ per batch with data-dependent control flow. The three-gate pipeline above instead pays $\Theta(s+m)$ for the union, $\Theta((s+m)\log(s+m))$ for LexSort and SegSum, $\Theta(s + m)$ for the threshold-residual certificate, and $\Theta(s)$ for ReplaceState — total $\Theta((s + m)\log(s + m))$ per batch, matching Table **??**. The control flow is fully data-independent given the public capacities (s, m).

H Constraint-Count Breakdown

We use the representative deployment $m = 1024$, $d = 7$, $w = 2^{16}$, $T \approx 7000$, $s, s' \le 2^{14}$, Halo2 IPA over Pasta cycles. The numbers are order-of-magnitude estimates intended to ground relative shares.

Commitment-scheme cost across regimes. Naive Merkle: $\sim T \cdot 2850 \approx 2 \times 10^7$ constraints—larger than every other line item combined. *Homomorphic-additive vector commitment* (Pattern A linear): $T \cdot 150 + O(1) \approx 1.05 \times 10^6$, a 15–30× improvement. *Batched IPA/KZG* (Pattern A max, HLL): $T + \log|S| \approx 7019$ multi-opening ops, in the 10^5–10^6 range. *Whole-state recommit* (Pattern B): one MSM of size $|S| \le s$ or s'.

Per-family closed-form cost equations. For Count-Min (Pattern A linear under HPP-LookupHash + homomorphic-additive commitment) the per-batch update cost is, as in §**??**,

$$C_{\text{prv}}^{\text{CM}} = m \cdot L_{\text{lkp}} + 2(md)\log(md) \cdot c_{\text{cmp}} + T \cdot (c_{\text{seg}} + c_{\text{patch}}) + c_{\text{lin}}.$$

For HyperLogLog (Pattern A max under batched IPA/KZG opening),

$$C_{\text{prv}}^{\text{HLL}} = m \cdot (L_{\text{lkp}} + c_{\text{lr}}) + 2m \log m \cdot c_{\text{cmp}} + T \cdot c_{\text{seg}} + (T + \log|S|) \cdot c_{\text{open}} + c_{\text{lin}},$$

where c_{lr} is the per-record LeadingRun gate constant and c_{open} is the batched-multi-opening per-row constant. For SpaceSaving (Pattern B with whole-state recommit on a state of bounded size s),

$$C_{\text{prv}}^{\text{SS}} = (s+m)c_{\text{union}} + 2(s+m)\log(s+m)c_{\text{cmp}} + (s+m)c_{\text{seg}} + sc_{\text{topS}} + sc_{\text{patch}} + c_{\text{lin}},$$

Table 11: Greenwald–Khanna update with $\log s'$ fusion rounds and whole-state recommit ($s' \approx 9.8 \times 10^4$, $m = 1024$).

Component	Constraints	Share
GK-InsertSort + InitDelta	350,000	23%
FuseTuple ($\lceil \log s' \rceil = 17$ rounds)	720,000	48%
StableCompact	80,000	5%
ReplaceState (MSM over $\lvert S \rvert$)	350,000	23%
Total	$\sim 1.5 \times 10^6$	100%

where c_{union} is the per-row SS-Union copy constant and c_{topS} is the threshold-residual per-row constant. For Greenwald–Khanna (Pattern B with $\log s'$ FuseTuple rounds),

$$C^{\text{GK}}_{\text{prv}} = (s'+m)c_{\text{insort}} + (s'+m)\log(s'+m)c_{\text{cmp}} + (s'+m)\log(s')c_{\text{fuse}} + s'c_{\text{patch}} + c_{\text{lin}},$$

where c_{insort} is the GK-InsertSort per-row constant, c_{fuse} is the FuseTuple per-pairwise-decision constant, and the $\log s'$ factor reflects the rounds of pairwise fusion (limit F1 of §**??**). The constants $\{c_{\text{cmp}}, c_{\text{seg}}, c_{\text{patch}}, c_{\text{lin}}, L_{\text{lkp}}, c_{\text{lr}}, c_{\text{open}}, c_{\text{union}}, c_{\text{topS}}, c_{\text{insort}}, c_{\text{fuse}}\}$ are calibrated against published Halo2 cost microbenchmarks [41] and yield the per-family numbers in Tables 8–10 below.

Table 8: Count-Min update with HPP and homomorphic-additive vector commitment.

Component	Constraints	Share
Hash verification (HPP lookup, $md = 7168$)	36,000	2%
Sort / permutation	750,000	40%
Segment aggregation	70,000	4%
Commitment patch	1,050,000	55%
Tree append	1,000	<1%
Total	$\sim 1.9 \times 10^6$	100%

Table 9: HyperLogLog update with batched IPA opening.

Component	Constraints	Share
Hash verification (m Poseidon + LeadingRun)	154,000	27%
Sort / permutation	100,000	18%
Segment max + sparse rebind	10,000	2%
Commitment opening	300,000	53%
Total	$\sim 5.6 \times 10^5$	100%

Table 10: SpaceSaving update with whole-state recommit ($s = m = 1024$).

Component	Constraints	Share
Buffer union (SS-Union)	30,000	6%
Sort + segment grouping	225,000	44%
Top-s certificate	100,000	20%
ReplaceState (MSM over $\lvert S \rvert$)	155,000	30%
Total	$\sim 5.1 \times 10^5$	100%

After applying the recommended schemes, no single component dominates by an order of magnitude; total cost lands in the 0.5–3.0 million constraint range. This is a 10–20× improvement over the naive Merkle baseline.

I Capacity Co-Sketch (CCS): DP-Bounded Capacity Privacy

zkSketch's three body-level compositional techniques — SDC, HPP, NDC — are engineering optimisations on the basic delta certificate that reduce cost without changing what leaks. The capacity co-sketch (CCS) is qualitatively different, because it changes *what leaks*: it converts the choice of public per-batch capacity from a deterministic, data-dependent quantity into a differentially private one. We describe the construction in detail here; the privacy-composition argument is then Proposition **??** in Appendix J.

Bottleneck. The padded capacities $T_{\max}$, s, s', G are public, and circuit obliviousness depends on this. If a deployment fixes capacities statically at worst-case values, padding overhead is large. If it instead adapts the capacity per batch to minimise padding, the choice of capacity itself leaks data-dependent quantities: touched-cell counts correlate with key skew, distinct-key cardinality with workload concentration, and so on.

Insight. Replace the deterministic adaptive capacity with a differentially private one. The next-batch capacity is then a DP-bounded function of the touched-cell history, and the public circuit shape becomes a function only of DP-released capacities. Circuit obliviousness composes with a quantifiable leakage budget instead of a vague best-effort hope, and the leakage joins the cost surface (§**??**) as a third coordinate alongside prover and verifier work. This lifts VPJoin's DP-guided capacity idea [2] from join-intermediate cardinalities to sketch touched-cell cardinalities.

Construction. CCS maintains a small auxiliary committed sketch $\widetilde{T}_t$ alongside the primary sketch. $\widetilde{T}_t$ tracks an estimator of the touched-cell cardinality of recent batches. The next batch's padded capacity is

$$T^{(t+1)}_{\max} = \text{ceil}\Big(\widetilde{T}_t + \text{Lap}\big(\tfrac{\Delta_T}{\varepsilon}\big)\Big),$$

where $\text{Lap}(\Delta_T/\varepsilon)$ is Laplace noise calibrated to the per-record contribution sensitivity Δ_T and the per-batch privacy budget ε. The sensitivity is the maximum number of distinct cells a single record can touch: $\Delta_T = d$ for Count-Min (one cell per row), $\Delta_T = 1$ for HyperLogLog (one register per record), $\Delta_T = 1$ for SpaceSaving (at most one promotion per record), and $\Delta_T = 2$ for Greenwald–Khanna (at most two adjacent rank intervals per record). To stop the prover from forging $\widetilde{T}_t$ adversarially, the CCS update gate binds the primary sketch and the auxiliary sketch with a coupling permutation: the touched-cell aggregate A used to update the primary must be the very same A used to update $\widetilde{T}_t$.

Savings. CCS converts an uncountable leak (an exact data-dependent capacity release) into a quantifiable one ($(\varepsilon_N, \delta_N)$-DP) at modest cost. The auxiliary sketch $\widetilde{T}_t$ is a Count-Min counter of dimensions $(d_{\text{cc}}, w_{\text{cc}}) = (3, 256)$ tracking touched-cell cardinality, with state size $\lvert\widetilde{T}_t\rvert = 768$ counters — about 0.2% of a representative primary CM state ($d = 7$, $w = 2^{16}$). The CCS update circuit adds an extra HashProfile evaluation per touched cell, an extra SegSum, and the coupling permutation; on the running CM deployment

this contributes ∼15K constraints, or ∼1% over the primary update. Composing N such per-batch capacity releases gives $(\varepsilon_N, \delta_N)$-differential privacy under standard composition (Proposition **??**), with advanced-composition tightening when N is large. The planner of §**??** treats the resulting bound as the leakage coordinate of the three-dimensional cost surface.

Soundness. The coupling permutation in the construction binds the auxiliary sketch to the same touched-cell aggregate A that the primary sketch updates against, so a prover cannot forge $\widetilde{T}_t$ adversarially without breaking the permutation argument. The DP guarantee is unconditional on prover honesty: the Laplace noise is sampled from a committed seed via the release-circuit mechanism of §3, and the seed is bound by M before any capacity is released, so the noise distribution is fixed independently of the data observed in any batch.

Capacity-policy trade-off. The three capacity policies span a continuous design space, summarised in Table 12. *Static* pads to the worst-case capacity once and forever; cost is high (the circuit always pays for the worst case) but leakage is zero (the public capacity reveals nothing about the data). *Adaptive* resizes the capacity per batch to track actual workload; cost drops by the padding ratio but every capacity release leaks an exact data-dependent quantity, so $\mathcal{L} = \infty$ under our obliviousness definition. *DP-via-CCS* sits between: the capacity is data-adaptive but DP-noised, so cost approaches adaptive in expectation while leakage is bounded by $(\varepsilon_N, \delta_N)$. The planner's choice depends on the deployment's leakage budget: regulated audit-grade deployments prefer Static (zero leakage at higher constant cost); throughput-grade deployments tolerate Adaptive when capacity itself is not sensitive (e.g., already-public touched-cell counts); leakage-tight deployments take DP-via-CCS as the principled middle ground.

Table 12: Capacity-policy trade-off. $T_{\mathbf{wc}}$ is the worst-case touched-cell count; $T_{\mathbf{actual}}$ is the realised value under the workload; $\rho = T_{\mathbf{actual}}/T_{\mathbf{wc}}$ is the padding ratio.

Policy	Per-batch padding cost	Leakage
Static	$\Theta(T_{\text{wc}})$	0
Adaptive	$\Theta(\rho \cdot T_{\text{wc}})$	∞ (exact value released)
DP-via-CCS	$\Theta(\rho \cdot T_{\text{wc}}) + \widetilde{O}(1)$	$(\varepsilon_N, \delta_N)$-DP

J Soundness Proofs

J.1 Update Soundness (Theorem 1)

Let $\mathcal{P}^*$ be a PPT prover producing an accepted update proof for x_t. The PLONKish extractor recovers w_t satisfying every circuit constraint. We argue w_t contains the required (S, B_t, S') via four claims. *(i) Faithful opening.* The witness contains openings of C_{old} and C_{B_t}; commitment binding makes any extracted opening unique. *(ii) Faithful delta generation.* The Generate gate enforces $\Delta = \text{Generate}_\tau(B_t)$ pointwise via polynomial constraints under the declared profile. *(iii) Faithful aggregation.* Permute, LexSort, Segment, and SegSum/SegMax force the sorted delta table to be a permutation of the generated table and the per-segment aggregate to be the correct sum/max per cell. *(iv) Faithful patch and composition extension.* The patch gate forces $C_{new} = \text{Com}(A_\tau.\text{Update}(S, B_t))$ uniquely by binding. The composition-extension gate (Chained-Fold, Tree-Append, or k-ary-Merge per the chosen topology of §4.3) forces $R^{(t)}$ exactly: the new composition root differs from any honestly-extended root only if the prover violates collision resistance or the recursive-SNARK knowledge soundness of the substrate. The conjunction is exactly $\mathcal{R}^\tau_{\text{upd}}$.

J.2 Framework-Level Closure (Theorem J.2)

Suppose every slot of Π_τ admits a sub-soundness lemma against the certificate catalog (Table **??**). Then the update circuit instantiated for τ is knowledge-sound for $\mathcal{R}^\tau_{\text{upd}}$. The proof follows from per-slot sub-soundness composed via the same extraction-then-binding argument as Theorem 1, with each slot's local invariant supplying one step of claim (ii)–(iv).

J.3 SDC Closure (Theorem J.3)

The SDC update circuit for K coupled sketches is knowledge-sound for the conjunction of K per-sketch update relations $\mathcal{R}^{\tau_k}_{\text{upd}}$, with one shared sort/permutation/tree-append and per-sketch patches. Tag-aware permutation ensures the sorted table cleanly separates the K sketches; per-sketch patch gates inherit per-sketch sub-soundness.

J.4 Workflow Soundness (Theorem ??)

The argument composes per-node soundness (per-relation extractors) with edge-commitment binding and recursive-SNARK composition-root binding (§4.3). For each edge $u \to v$ carrying boundary commitment Com, the equality of u's output and v's input commitment under the verifying check, plus binding, implies the underlying values are equal. The composition root commitment $R^{(t)}$ then forces the temporal aggregation order: any reordering of subproofs produces a different root under whichever composition topology (chained, binary-tree, or k-ary) is in use, contradicting the public-input fix and the underlying recursive-SNARK knowledge soundness. The argument is uniform across topologies because each delivers a single binding root with the same soundness profile.

J.5 CCS Privacy Composition (Proposition ??)

The CCS releases the public capacity $T^{(t+1)}_{\max} = \text{ceil}(\widetilde{T}_t + \text{Lap}(\Delta_T/\varepsilon))$ once per batch, where Δ_T bounds the per-record contribution to touched-cell count. The composition of N such per-batch releases is $(\varepsilon_N, \delta_N)$-differentially private under standard sequential composition; under advanced composition for the Laplace mechanism, ε_N grows as $O(\sqrt{N \log(1/\delta_N)} \cdot \varepsilon)$ rather than $N \cdot \varepsilon$. A deployment chooses the per-batch ε to fit a global per-audit-window budget $(\varepsilon_N, \delta_N)$; the planner of §**??** treats the resulting bound as the leakage coordinate of the three-dimensional cost surface.

K Future Extensions

The chained, binary-tree, and k-ary composition topologies of §4.3 cover the recursive-SNARK composition design space at the framework level. Two further directions remain open. *Hybrid topologies*: mixing chained folding within k-ary leaves — a batched leaf of size m runs an internal folding chain over its m subproofs, then participates in a k-ary tree — combines the per-batch latency of

folding with the parallelism of trees. We leave a careful evaluation to future work. *Adaptive arity*: dynamically tuning k across tree levels to match the prover's available cores at each layer (high k near the root where parallel work is exhausted; low k at the leaves where per-merge circuits are cheap) is a natural compiler extension.